# MERN Stack Workshop

Covering elementary concepts behind a basic MERN project

aadilmohammed360.github.io →



# Credits

Instructor: Aadil

Supervisor: Prof. Rajitha

# Introduction

→ In this presentation, we will walk you through setting up your MERN development environment from scratch on Windows. Youll learn how to organize project folders, install Node.js and MongoDB, and troubleshoot common installation issues. Well also cover running the backend and frontend servers to get your project up and running smoothly.

# Topics Covered

1. Installation of tools for MERN development
2. Verifying error free installation and troubleshooting with fixes
3. Setting the development environment and folder structure
4. Learning how to use Mongodb Compass to insert records in database and enforce strict schema using Mongoose
5. Programming Express backend to communicate with backend and perform CRUD operations
6. Building Frontend Interface using React to control backend and manipulate database using forms and buttons

# Workshop Materials

1. Blog at https://aadilmohammed360.github.io/Learn-MERN-Fundamentals/
2. Github Repo with Final Source code at https://github.com/AadilMohammed360/Learn-MERN-Fundamentals

# What is MERN Stack ?

Consider a builders a toolbox , it wil have various tools like hammer, spanner, saw, screwdriver etc

Similarly, MERN stack is a bundle of tools (toolbox), where each letter corresponds to a single tool with a specialized purpose

# What is MERN Stack ?




Stands for Mongo Database




Stands for Express Backend

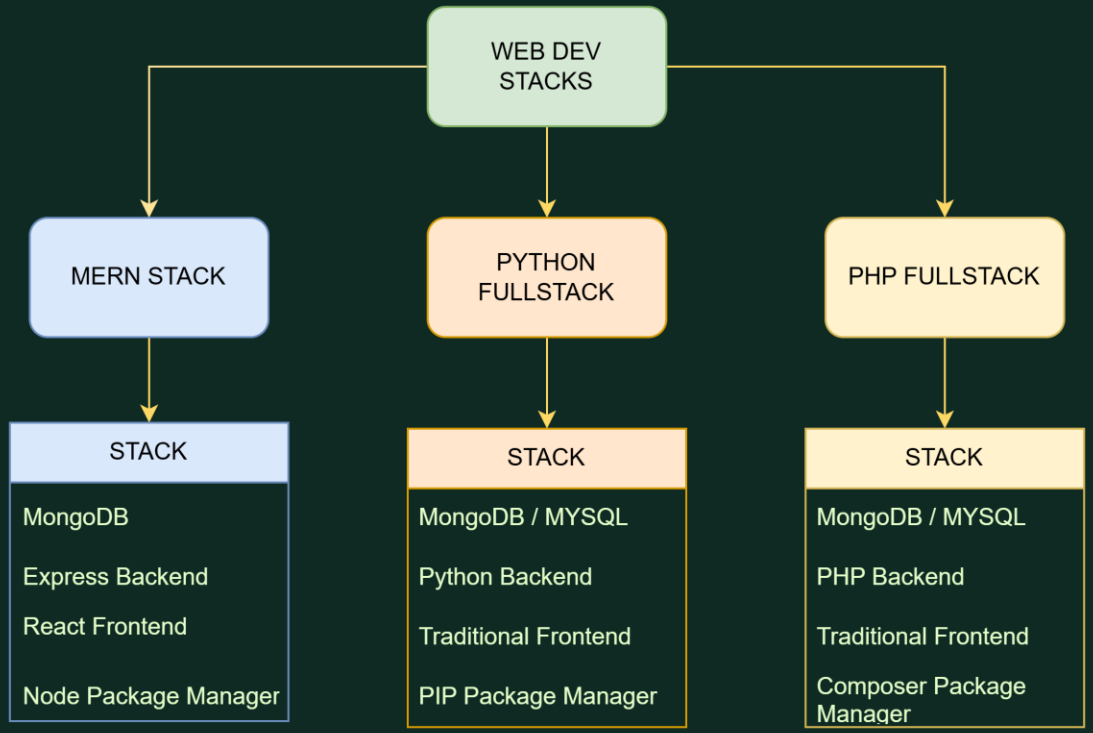



Stands for React Frontend




Stands for Node Environment & Package Manager

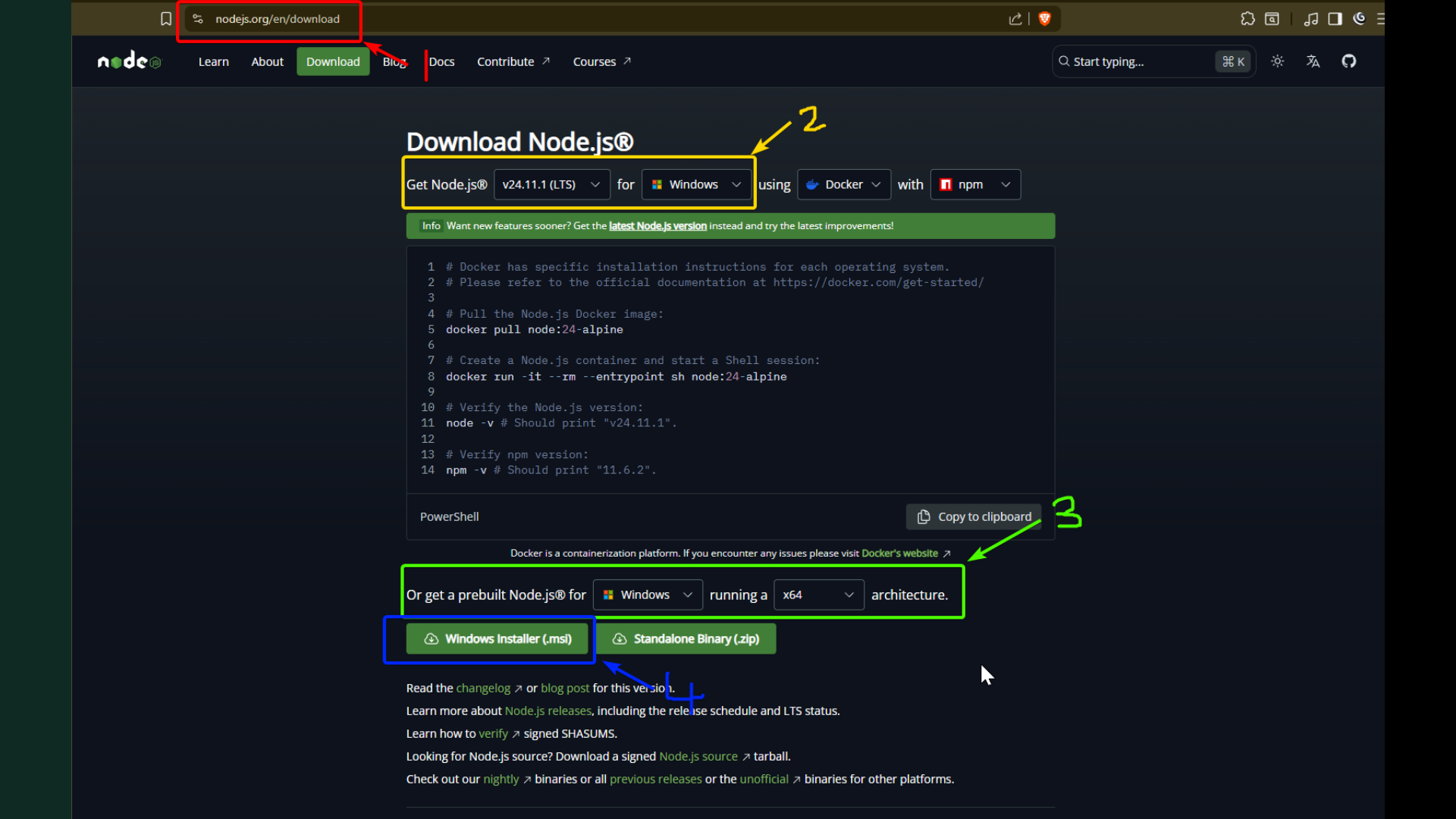
# Full Stack Path Choices

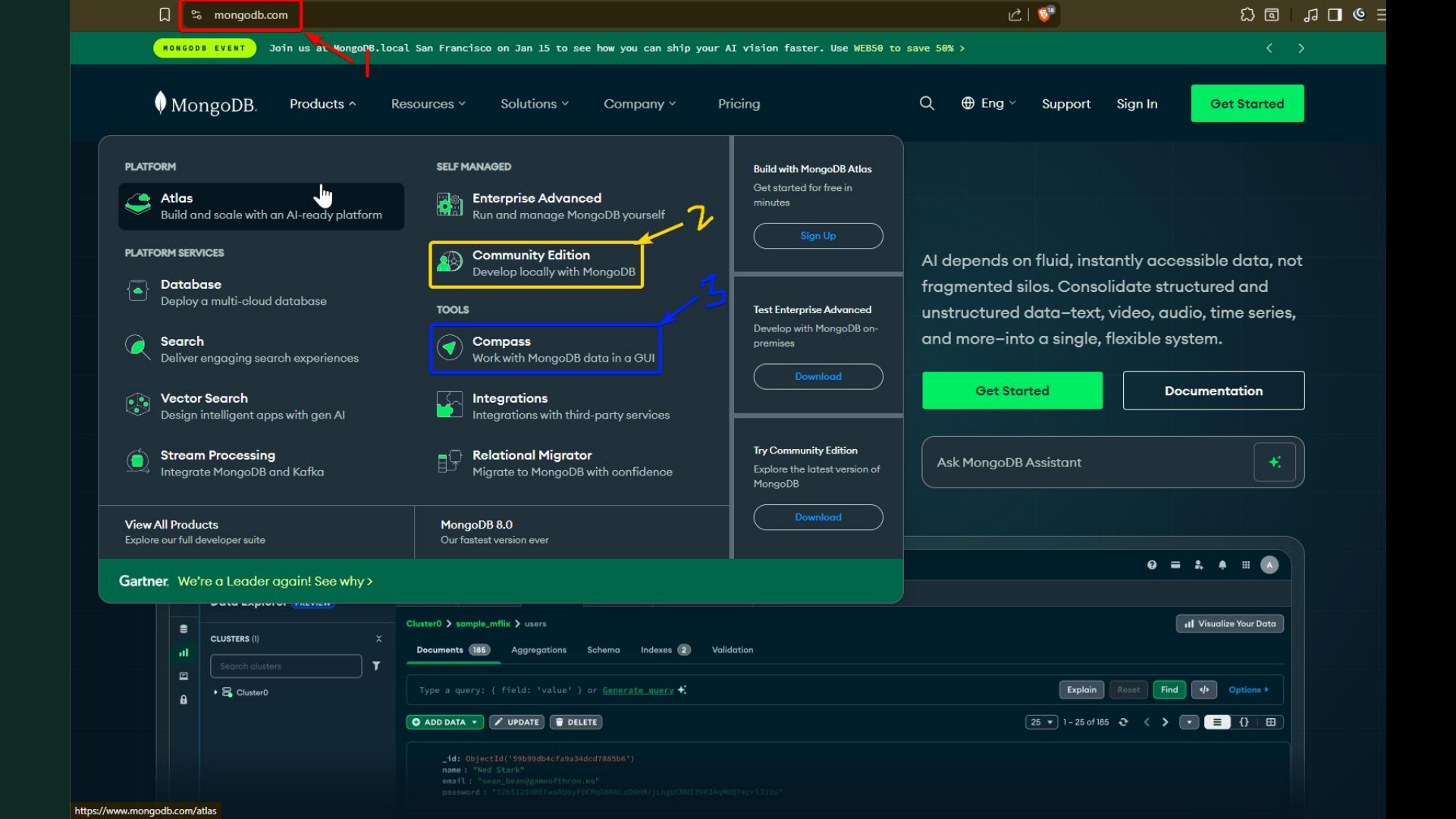


# MERN Setup Prerequisites

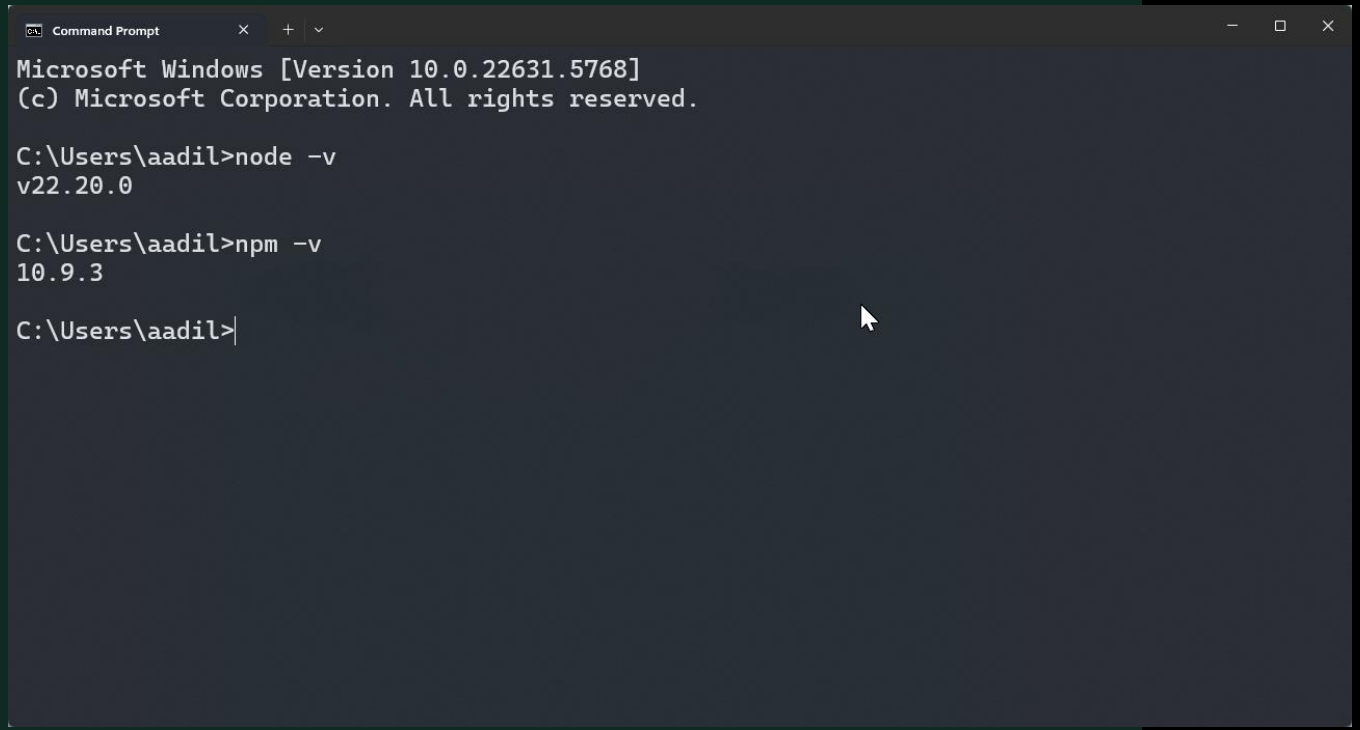
- Install node js from nodejs.org
- Install mongodb community edition
- Install mongodb compass




nodejs.org/en/download
Learn
About
Download
Blog
Docs
Contribute
Courses
Start typing...
⌘ K
Download Node.js®
2
Get Node.js® v24.11.1 (LTS) for Windows using Docker with npm
Info Want new features sooner? Get the latest Node.js version instead and try the latest improvements!
1 # Docker has specific installation instructions for each operating system.
2 # Please refer to the official documentation at https://docker.com/get-started/
3
4 # Pull the Node.js Docker image:
5 docker pull node:24-alpine
6
7 # Create a Node.js container and start a Shell session:
8 docker run -it --rm --entrypoint sh node:24-alpine
9
10 # Verify the Node.js version:
11 node -v # Should print "v24.11.1".
12
13 # Verify npm version:
14 npm -v # Should print "11.6.2".
PowerShell
Copy to clipboard
3
Docker is a containerization platform. If you encounter any issues please visit Docker's website
Or get a prebuilt Node.js® for Windows running a x64 architecture.
Windows Installer (.msi)
Standalone Binary (.zip)
4
Read the changelog or blog post for this version.
Learn more about Node.js releases, including the release schedule and LTS status.
Learn how to verify signed SHASUMS.
Looking for Node.js source? Download a signed Node.js source tarball.
Check out our nightly binaries or all previous releases or the unofficial binaries for other platforms.


mongodb.com
MONGODB EVENT
Join us at MongoDB.local San Francisco on Jan 15 to see how you can ship your AI vision faster. Use WEB50 to save 50% >
MongoDB.
Products
Resources
Solutions
Company
Pricing
Eng
Support
Sign In
Get Started
PLATFORM
Atlas
Build and scale with an AI-ready platform
PLATFORM SERVICES
Database
Deploy a multi-cloud database
Search
Deliver engaging search experiences
Vector Search
Design intelligent apps with gen AI
Stream Processing
Integrate MongoDB and Kafka
SELF MANAGED
Enterprise Advanced
Run and manage MongoDB yourself
Community Edition
Develop locally with MongoDB
TOOLS
Compass
Work with MongoDB data in a GUI
Integrations
Integrations with third-party services
Relational Migrator
Migrate to MongoDB with confidence
Build with MongoDB Atlas
Get started for free in minutes
Sign Up
Test Enterprise Advanced
Develop with MongoDB on-premises
Download
Try Community Edition
Explore the latest version of MongoDB
Download
View All Products
Explore our full developer suite
MongoDB 8.0
Our fastest version ever
Gartner We're a Leader again! See why >
2
3
AI depends on fluid, instantly accessible data, not fragmented silos. Consolidate structured and unstructured data–text, video, audio, time series, and more–into a single, flexible system.
Get Started
Documentation
Ask MongoDB Assistant
CLUSTERS (1)
Search clusters
Cluster0
Cluster0 > sample_mflix > users
Visualize Your Data
Documents 185
Aggregations
Schema
Indexes 2
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
UPDATE
DELETE
25
1 - 25 of 185
_id: ObjectId('59b99db4cfa9a34dcd7885b6')
name : "Ned Stark"
email : "sean_bean@gameofthrones.es"
password : "$2b$12$U8EFwsRUoyF8CRqGNK6Lz0HH/1LhpDCNNI39RJAqNUQ74crlJ1Vu"
https://www.mongodb.com/atlas

# Checking NodeJS Installated Correctly



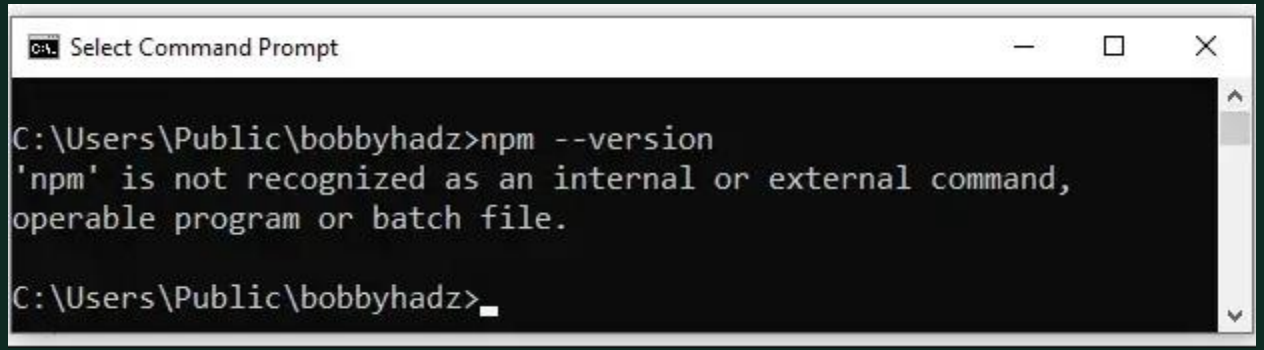
# Node.js, npm, and MongoDB Installation Fixes

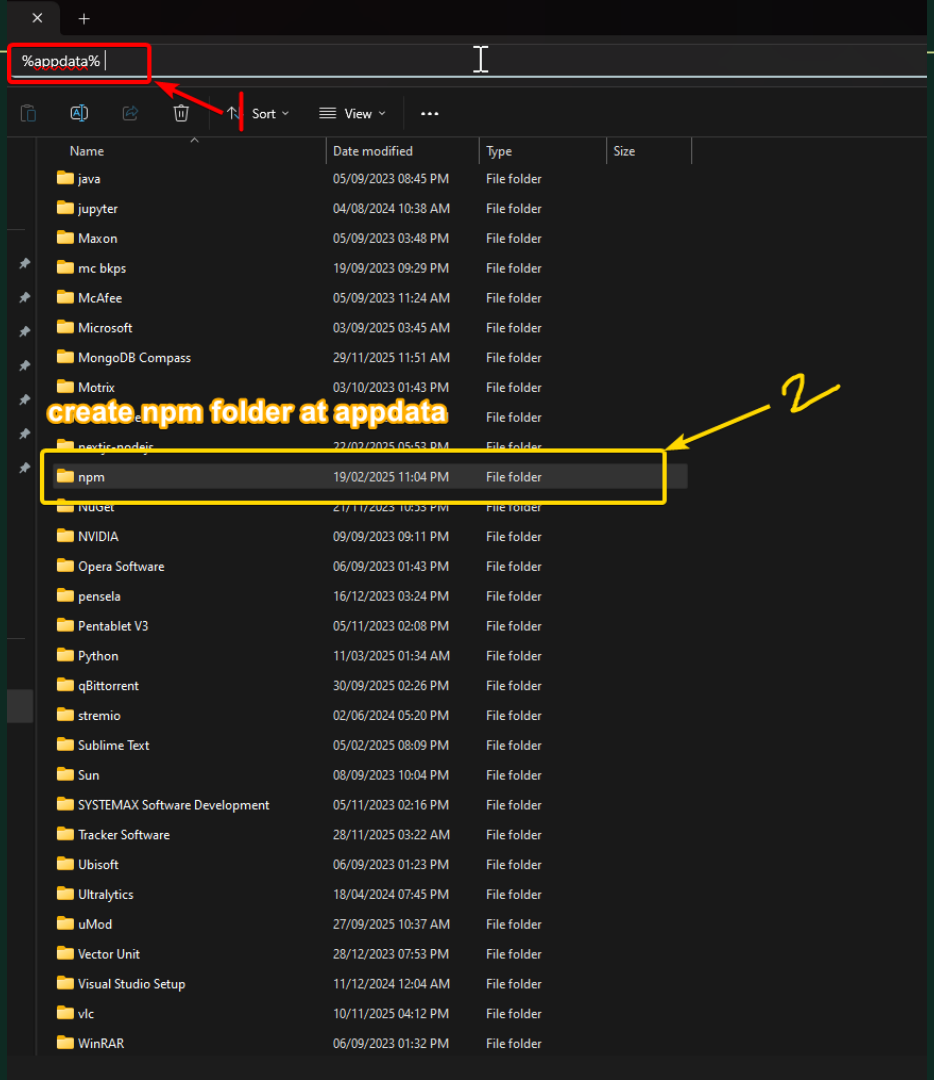
→ Sometimes npm scripts are blocked by PowerShell, or npm and Node.js commands fail due to missing folders or PATH misconfiguration. Fix these by adjusting execution policies, creating necessary folders, and adding Node.js paths to your system environment variables. Confirm installation success by running node -v and npm -v commands.

## ✅ npm Errors & Solutions — Summary Table

| Error Type | Example Error Message | Cause | Solution |
|---|---|---|---|
| **1. npm not found** | `npm: command not found` or `npm is not recognized as an internal or external command` | npm folder missing from PATH, or npm not installed correctly | • Add this to **User PATH:** `%AppData%\npm`<br>• Add this to **System PATH:** `C:\Program Files\nodejs\`<br>• Reinstall Node.js if npm files are missing |
| **2. Cannot find npm-cli.js** | `Cannot find module '...\node_modules\npm\bin\npm-cli.js'` | PATH or npm prefix points to the wrong (nonexistent) npm directory | • Remove incorrect PATH entries like `node_modules\npm\bin`<br>• Ensure npm exists in `C:\Program Files\nodejs\`<br>• Fix npm prefix: `npm config set prefix "%APPDATA%\npm"` |
| **3. PowerShell scripts disabled** | `Running scripts is disabled on this system` | PowerShell's Execution Policy prevents `.ps1` scripts (e.g., npm, npx) from running | • Run PowerShell as Admin: `Set-ExecutionPolicy RemoteSigned`<br>OR for user only: `Set-ExecutionPolicy RemoteSigned -Scope CurrentUser`<br>OR temporary: `Set-ExecutionPolicy Bypass -Scope Process` |

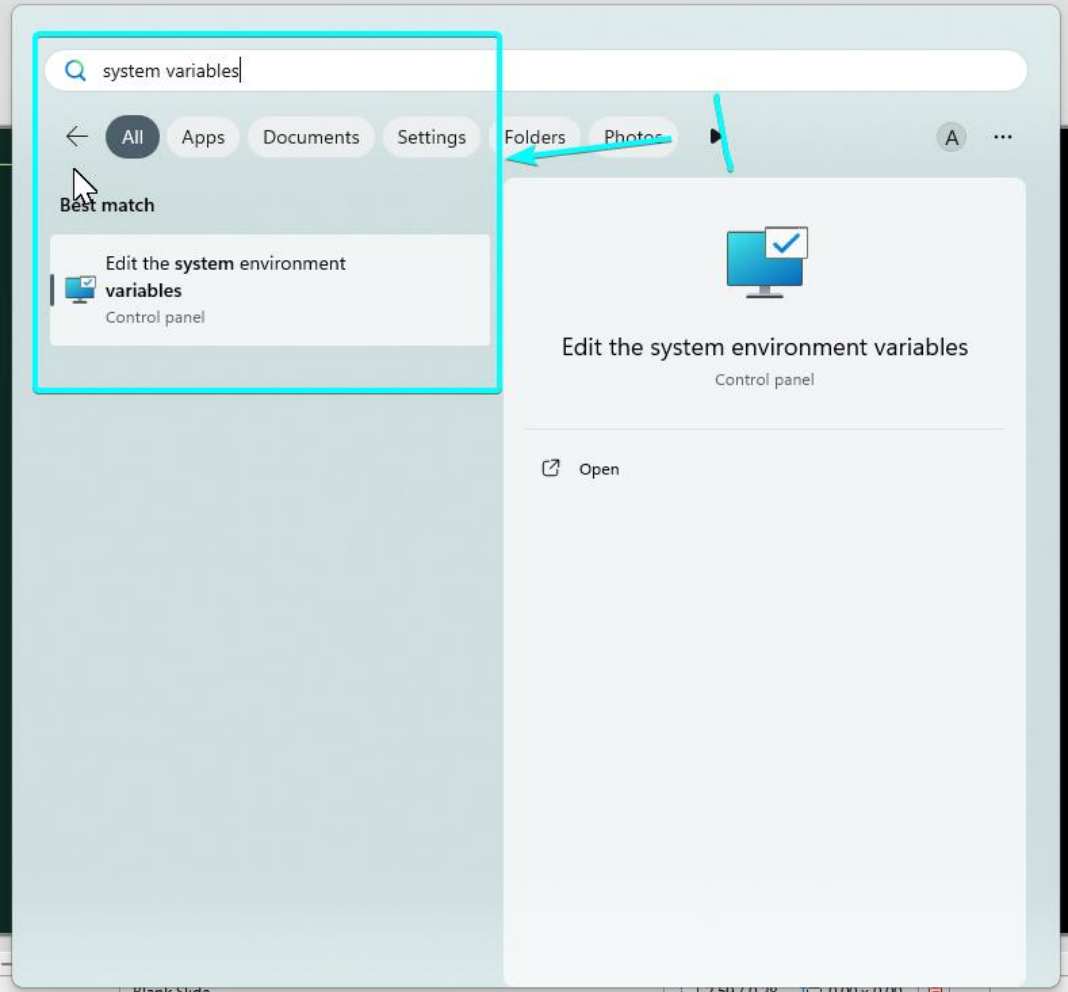
# Error 1: npm not found or recognised

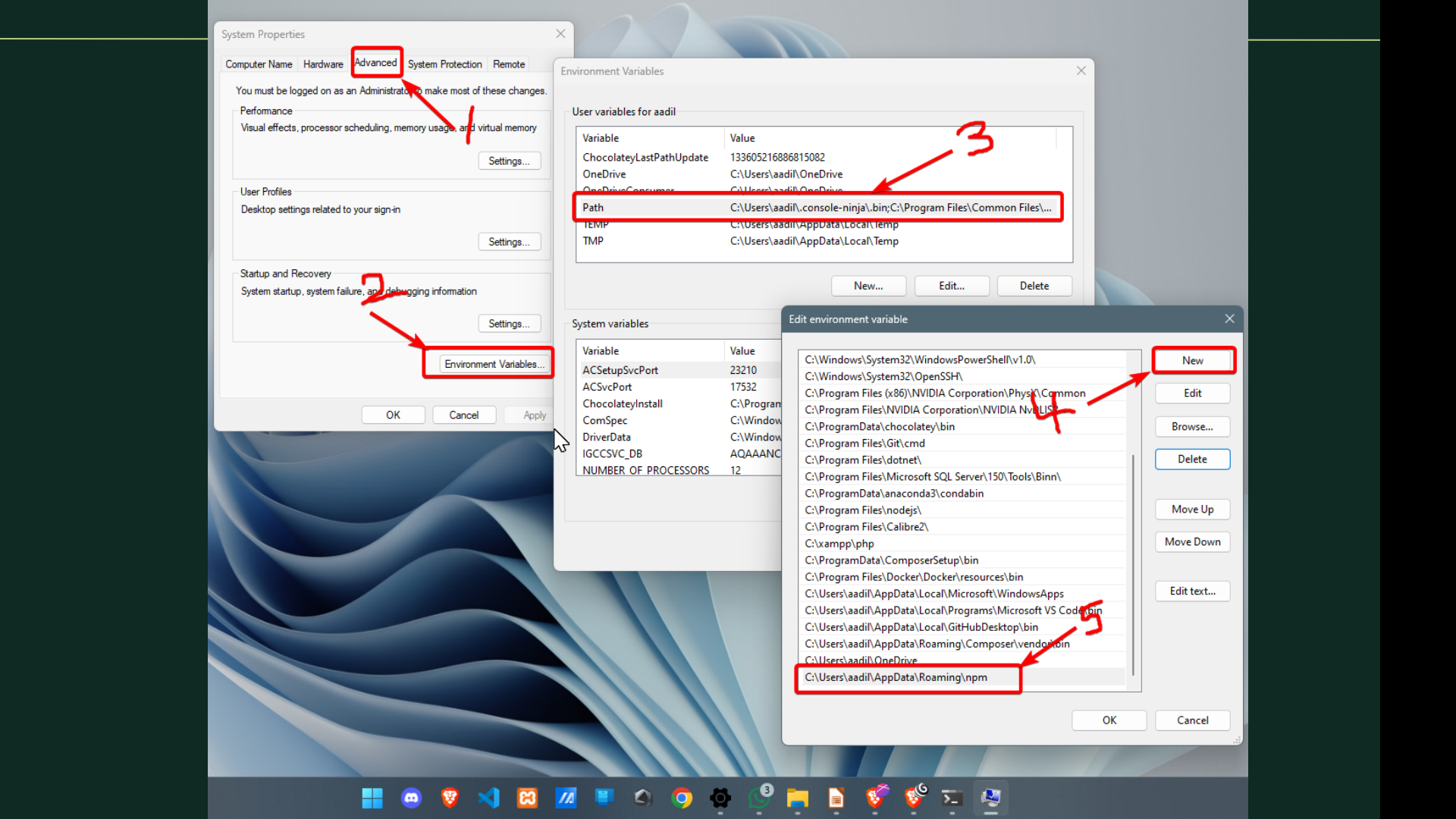



%appdata%
Sort
View
Name
Date modified
Type
Size
java 05/09/2023 08:45 PM File folder
jupyter 04/08/2024 10:38 AM File folder
Maxon 05/09/2023 03:48 PM File folder
mc bkps 19/09/2023 09:29 PM File folder
McAfee 05/09/2023 11:24 AM File folder
Microsoft 03/09/2025 03:45 AM File folder
MongoDB Compass 29/11/2025 11:51 AM File folder
Motrix 03/10/2023 01:43 PM File folder
create npm folder at appdata
File folder
npm 19/02/2025 11:04 PM File folder
NVIDIA 09/09/2023 09:11 PM File folder
Opera Software 06/09/2023 01:43 PM File folder
pensela 16/12/2023 03:24 PM File folder
Pentablet V3 05/11/2023 02:08 PM File folder
Python 11/03/2025 01:34 AM File folder
qBittorrent 30/09/2025 02:26 PM File folder
stremio 02/06/2024 05:20 PM File folder
Sublime Text 05/02/2025 08:09 PM File folder
Sun 08/09/2023 10:04 PM File folder
SYSTEMAX Software Development 05/11/2023 02:16 PM File folder
Tracker Software 28/11/2025 03:22 AM File folder
Ubisoft 06/09/2023 01:23 PM File folder
Ultralytics 18/04/2024 07:45 PM File folder
uMod 27/09/2025 10:37 AM File folder
Vector Unit 28/12/2023 07:53 PM File folder
Visual Studio Setup 11/12/2024 12:04 AM File folder
vlc 10/11/2025 04:12 PM File folder
WinRAR 06/09/2023 01:32 PM File folder
2

```
F:\All Work\Js Material\TraversyMedia Work\React Projects\dev_portfolio>npm -v
internal/modules/cjs/loader.js:883
  throw err;
  ^

Error: Cannot find module 'C:\Program Files\nodejs\node_modules\npm\bin\node_modules\npm\bin\npm-cli.js'
    at Function.Module._resolveFilename (internal/modules/cjs/loader.js:880:15)
    at Function.Module._load (internal/modules/cjs/loader.js:725:27)
    at Function.executeUserEntryPoint [as runMain] (internal/modules/run_main.js:72:12)
    at internal/main/run_main_module.js:17:47 {
  code: 'MODULE_NOT_FOUND',
  requireStack: []
}
```


system variables
All
Apps
Documents
Settings
Folders
Photos
Best match
Edit the system environment variables
Control panel
Edit the system environment variables
Control panel
Open


System Properties
Computer Name
Hardware
Advanced
System Protection
Remote
You must be logged on as an Administrator to make most of these changes.
Performance
Visual effects, processor scheduling, memory usage, and virtual memory
Settings...
User Profiles
Desktop settings related to your sign-in
Settings...
Startup and Recovery
System startup, system failure, and debugging information
Settings...
Environment Variables...
OK
Cancel
Apply
Environment Variables
User variables for aadil
Variable
Value
ChocolateyLastPathUpdate
133605216886815082
OneDrive
C:\Users\aadil\OneDrive
Path
C:\Users\aadil\.console-ninja\.bin;C:\Program Files\Common Files\...
TEMP
C:\Users\aadil\AppData\Local\Temp
TMP
C:\Users\aadil\AppData\Local\Temp
New...
Edit...
Delete
System variables
Variable
Value
ACSetupSvcPort
23210
ACSvcPort
17532
ChocolateyInstall
C:\Program
ComSpec
C:\Window
DriverData
C:\Window
IGCCSVC_DB
AQAAANC
NUMBER_OF_PROCESSORS
12
Edit environment variable
C:\Windows\System32\WindowsPowerShell\v1.0\
C:\Windows\System32\OpenSSH\
C:\Program Files (x86)\NVIDIA Corporation\PhysX\Common
C:\Program Files\NVIDIA Corporation\NVIDIA NvLiS
C:\ProgramData\chocolatey\bin
C:\Program Files\Git\cmd
C:\Program Files\dotnet\
C:\Program Files\Microsoft SQL Server\150\Tools\Binn\
C:\ProgramData\anaconda3\condabin
C:\Program Files\nodejs\
C:\Program Files\Calibre2\
C:\xampp\php
C:\ProgramData\ComposerSetup\bin
C:\Program Files\Docker\Docker\resources\bin
C:\Users\aadil\AppData\Local\Microsoft\WindowsApps
C:\Users\aadil\AppData\Local\Programs\Microsoft VS Code\bin
C:\Users\aadil\AppData\Local\GitHubDesktop\bin
C:\Users\aadil\AppData\Roaming\Composer\vendor\bin
C:\Users\aadil\OneDrive
C:\Users\aadil\AppData\Roaming\npm
New
Edit
Browse...
Delete
Move Up
Move Down
Edit text...
OK
Cancel
1
2
3
4
5

```
npm : File C:\Program Files\nodejs\npm.ps1 cannot be loaded because running
this system. For more information, see about_Execution_Policies at
https:/go.microsoft.com/fwlink/?LinkID=135170.
At line:1 char:1
+ npm run dev
+ ~~~
    + CategoryInfo          : SecurityError: (:) [], PSSecurityException
    + FullyQualifiedErrorId : UnauthorizedAccess
```

## ✅ Fix: Allow PowerShell to run local scripts

Open **PowerShell as Administrator** and run:

```powershell
Set-ExecutionPolicy RemoteSigned
```

Choose **Y** when prompted.

### ✔ What this does:

- Allows **local scripts** to run
- Blocks **unsigned scripts downloaded from the internet**

This is the safest common setting for development.

---

## 🔧 Alternative (if you want to keep system policy strict)

If you don't want to change the system policy, you can change **just for your user**:

```powershell
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
```

This is usually enough to fix npm errors.

---

## 🔧 Temporary workaround (per session only)

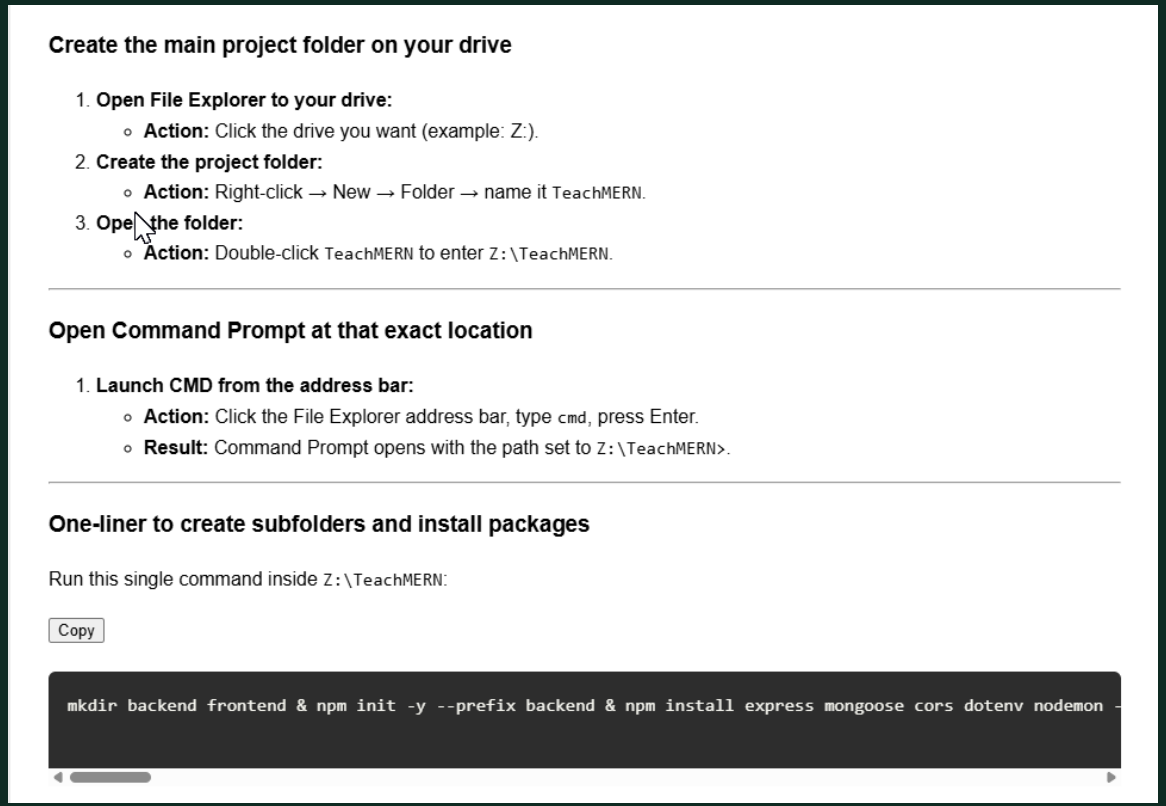
If you don't want to change the policy permanently, run:

```powershell
Set-ExecutionPolicy Bypass -Scope Process
```

# Windows Setup and Folder Structure

→ Start by creating a main project folder, for example, StudCRUD, on your drive for ex: Z: . Open Command Prompt within this folder by typing cmd in File Explorers address bar. Use a single-line command to create backend and frontend subfolders, initialize backend with package.json, install dependencies, and prepare scripts to run both servers efficiently.

- Goto: https://aadilmohammed360.github.io/Learn-MERN-Fundamentals/
- Copy my one liner command from chapter 1E



### Create the main project folder on your drive

1. **Open File Explorer to your drive:**
   - **Action:** Click the drive you want (example: Z:).
2. **Create the project folder:**
   - **Action:** Right-click → New → Folder → name it `TeachMERN`.
3. **Open the folder:**
   - **Action:** Double-click `TeachMERN` to enter `Z:\TeachMERN`.

---

### Open Command Prompt at that exact location

1. **Launch CMD from the address bar:**
   - **Action:** Click the File Explorer address bar, type `cmd`, press Enter.
   - **Result:** Command Prompt opens with the path set to `Z:\TeachMERN>`.

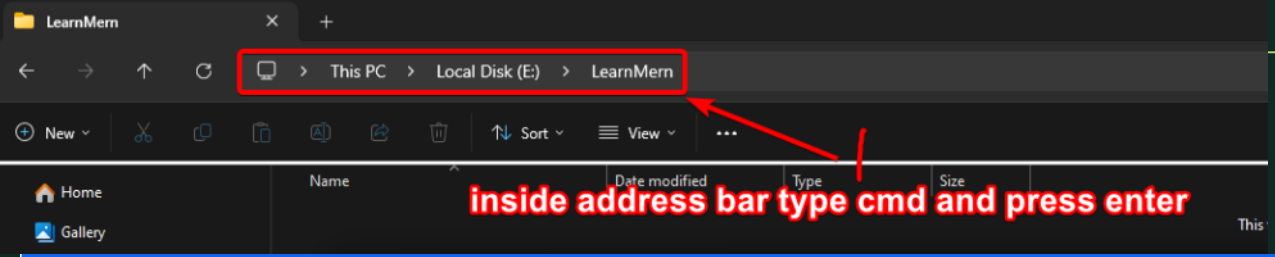
---

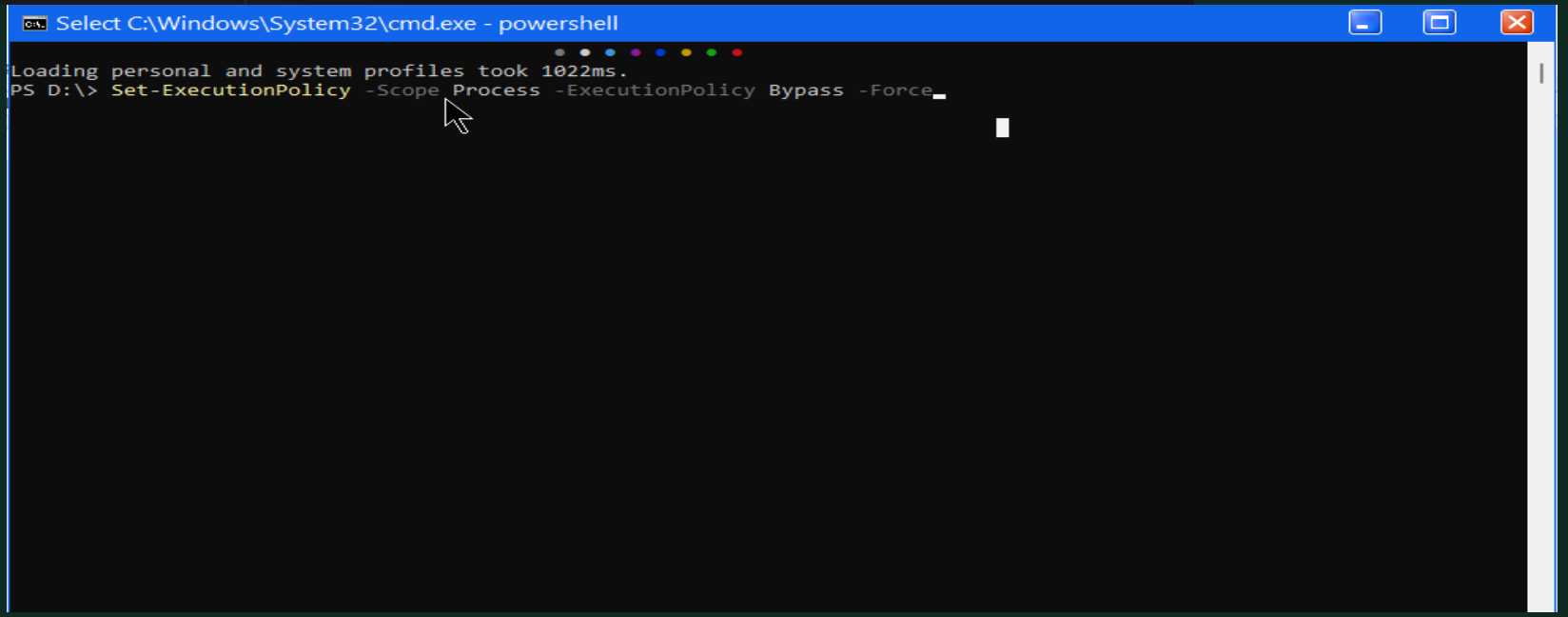
### One-liner to create subfolders and install packages

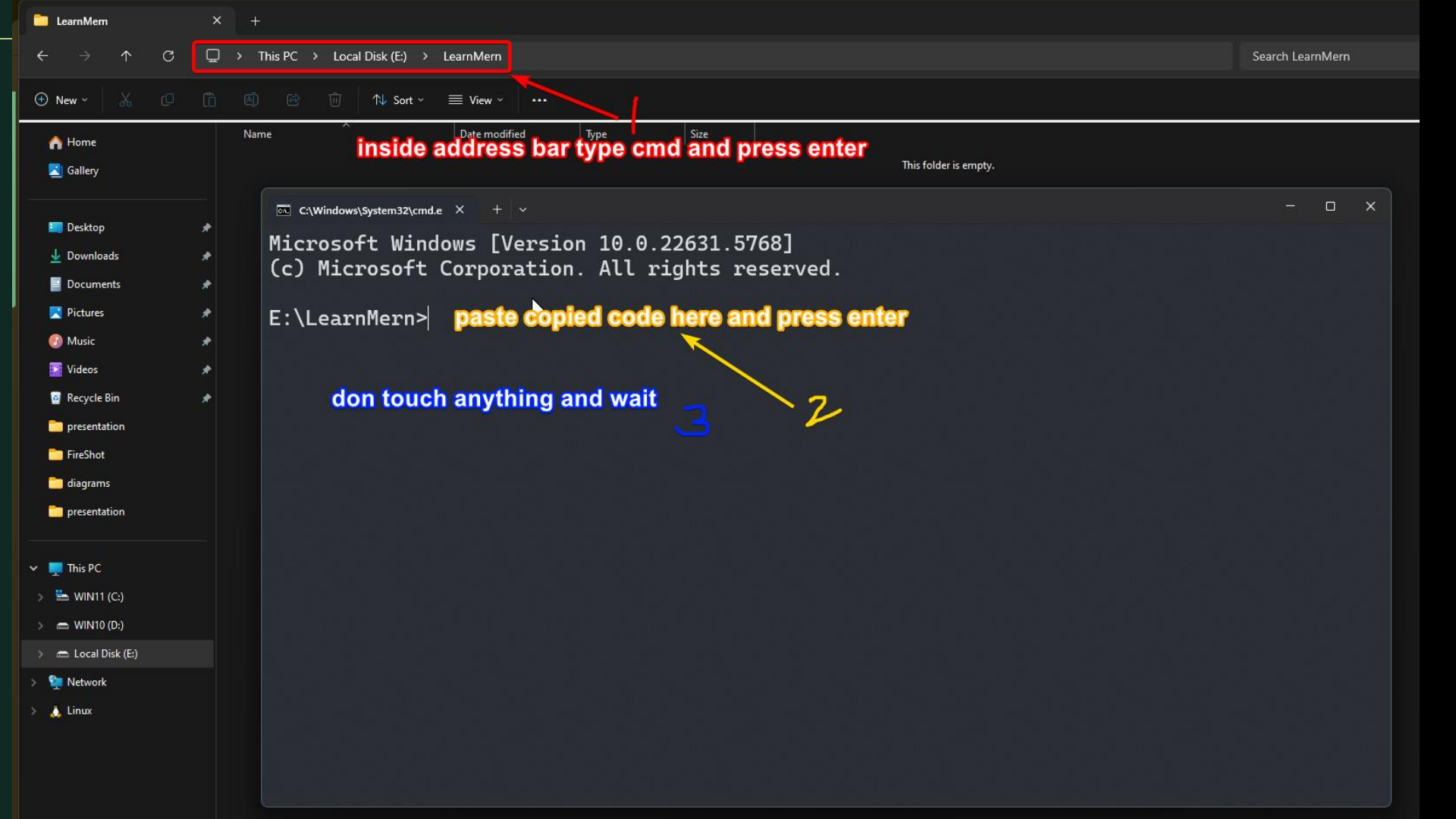
Run this single command inside `Z:\TeachMERN`:

Copy

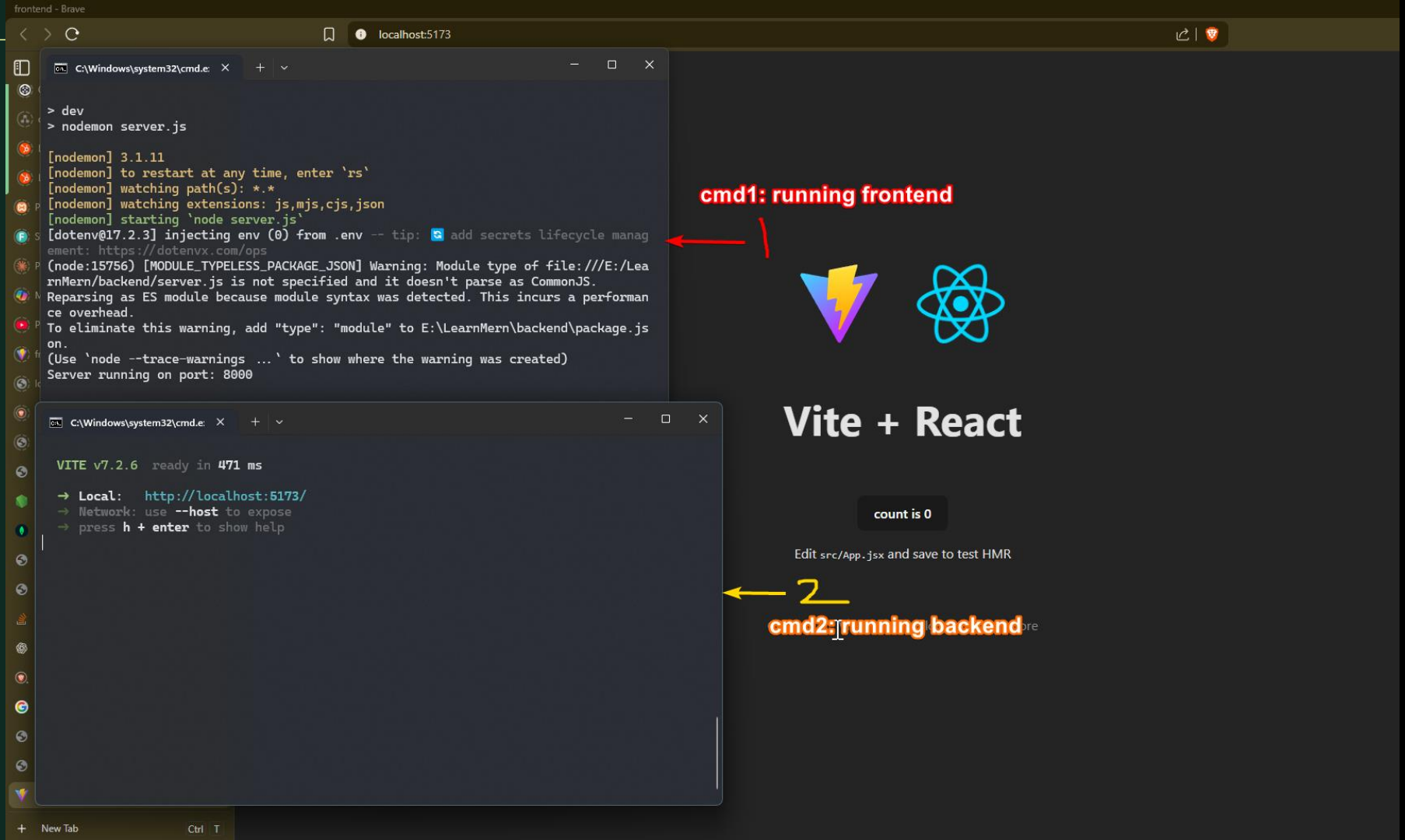
```
mkdir backend frontend & npm init -y --prefix backend & npm install express mongoose cors dotenv nodemon -
```

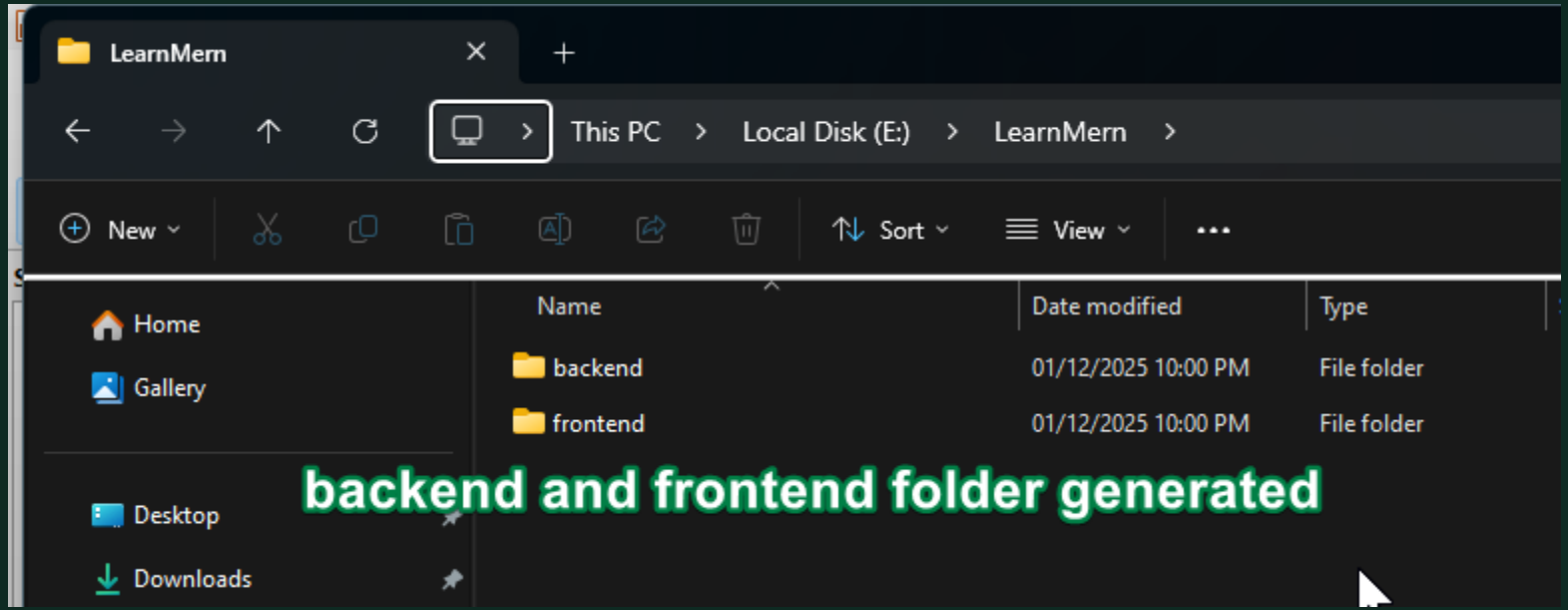

LearnMern
This PC
Local Disk (E:)
LearnMern
New
Sort
View
Home
Gallery
Name
Date modified
Type
Size
inside address bar type cmd and press enter


Select C:\Windows\System32\cmd.exe - powershell
Loading personal and system profiles took 1022ms.
PS D:\> Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force


LearnMern
This PC
Local Disk (E:)
LearnMern
Search LearnMern
New
Sort
View
Name
Date modified
Type
Size
inside address bar type cmd and press enter
1
This folder is empty.
Home
Gallery
Desktop
Downloads
Documents
Pictures
Music
Videos
Recycle Bin
presentation
FireShot
diagrams
presentation
This PC
WIN11 (C:)
WIN10 (D:)
Local Disk (E:)
Network
Linux
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.22631.5768]
(c) Microsoft Corporation. All rights reserved.
E:\LearnMern>
paste copied code here and press enter
2
don touch anything and wait
3

- My custom code will automatically set both backend and frontend folder setup for you
- 2 cmd windows will appear and they will run backend and frontend
- Dont  do anything, just wait until your browser opens with a react logo page and


frontend - Brave
localhost:5173
C:\Windows\system32\cmd.e
> dev
> nodemon server.js
[nodemon] 3.1.11
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
[dotenv@17.2.3] injecting env (0) from .env -- tip: add secrets lifecycle management: https://dotenvx.com/ops
(node:15756) [MODULE_TYPELESS_PACKAGE_JSON] Warning: Module type of file:///E:/LearnMern/backend/server.js is not specified and it doesn't parse as CommonJS.
Reparsing as ES module because module syntax was detected. This incurs a performance overhead.
To eliminate this warning, add "type": "module" to E:\LearnMern\backend\package.json.
(Use `node --trace-warnings ...` to show where the warning was created)
Server running on port: 8000
cmd1: running frontend
C:\Windows\system32\cmd.e
VITE v7.2.6 ready in 471 ms
➜ Local: http://localhost:5173/
➜ Network: use --host to expose
➜ press h + enter to show help
2
cmd2: running backend
Vite + React
count is 0
Edit src/App.jsx and save to test HMR
New Tab Ctrl T


LearnMern
This PC
Local Disk (E:)
LearnMern
New
Sort
View
Home
Gallery
Desktop
Downloads
Name
Date modified
Type
backend
01/12/2025 10:00 PM
File folder
frontend
01/12/2025 10:00 PM
File folder
backend and frontend folder generated

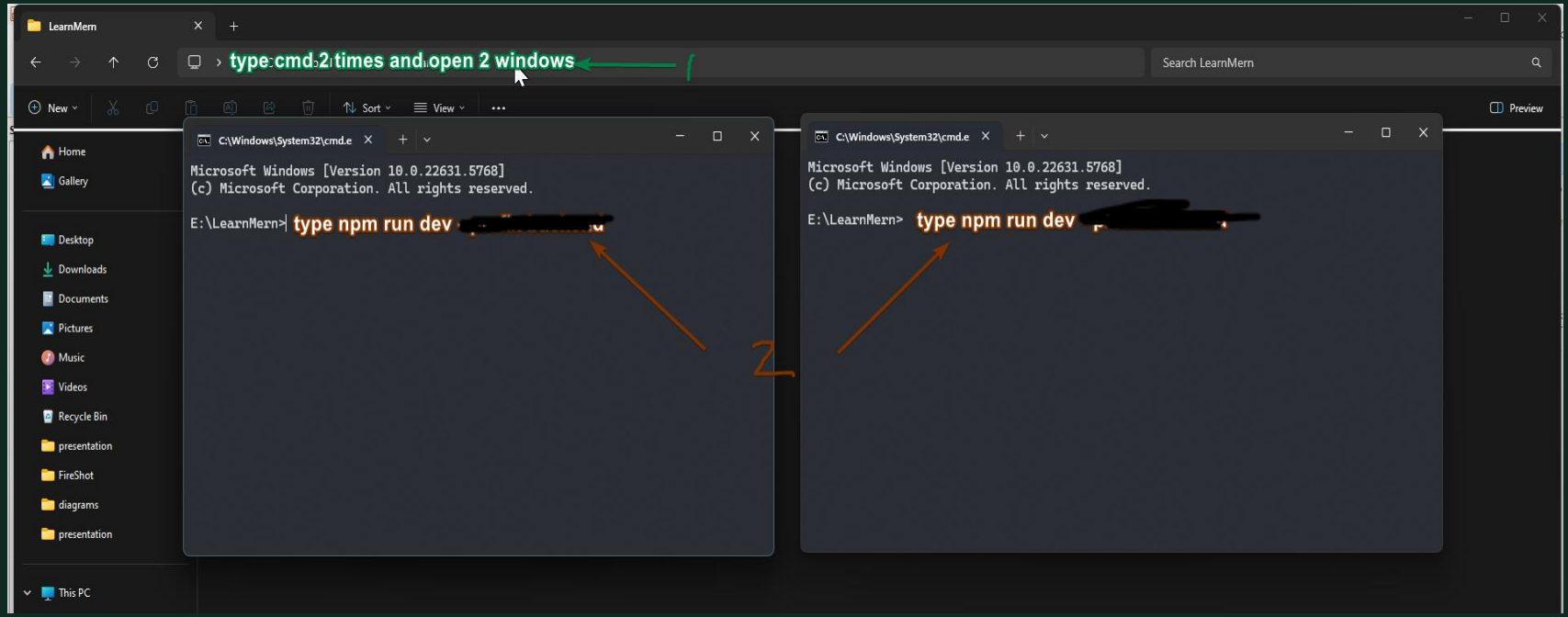
# Running Backend and Frontend Servers

Run the backend server with npm run dev --prefix backend and the frontend React app with npm run dev --prefix frontend in separate Command Prompts from address bar at Z:\StudCRUD. The backend typically runs on http://localhost:8000 and the frontend on http://localhost:5173. This setup supports live reloading to speed up development.

↓



# How to run the project next time



## Steps to Set Up a Vite Project

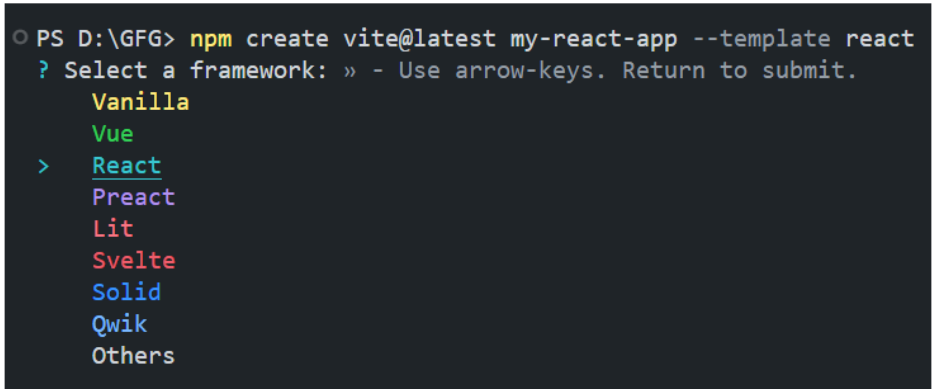
**Step 1:** Install NodeJs, If you haven't installed NodeJs, download it from the official website.

**Step 2:** Create a new Vite Project

```
npm create vite@latest my-react-app --template react
```

**Step 3:** Select a framework: select the **React** framework here using the downward arrow key. You can select any of the frameworks as per your requirements.

```
Vanilla
Vue
React
Preact
Lit
Svelte
Solid
Qwik
Others
```



Svelte
Solid
Qwik
Others

```
PS D:\GFG> npm create vite@latest my-react-app --template react
? Select a framework: » - Use arrow-keys. Return to submit.
    Vanilla
    Vue
>   React
    Preact
    Lit
    Svelte
    Solid
    Qwik
    Others
```

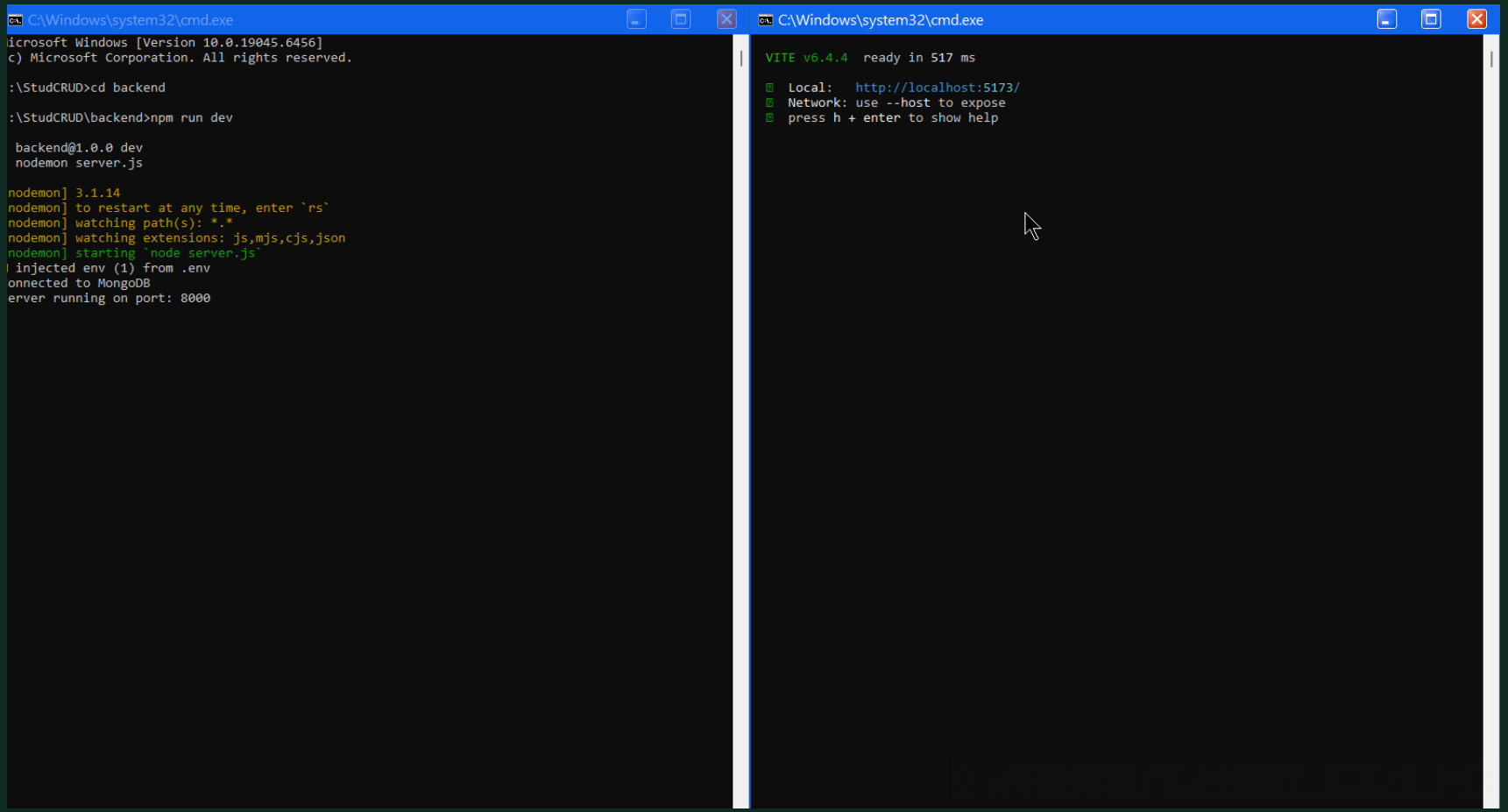
*select React*

**Step 4:** Select Variant: choose any variant of your choice using the downward arrow key,i.e: choose **JavaScript**.

TypeScript
TypeScript + SWC
**JavaScript**
JavaScript + SWC

```
? Select a variant: » - Use arrow-keys. Return to submit.
    TypeScript
    TypeScript + SWC
>   JavaScript
    JavaScript + SWC
    Remix ↗
```

*select javascript*

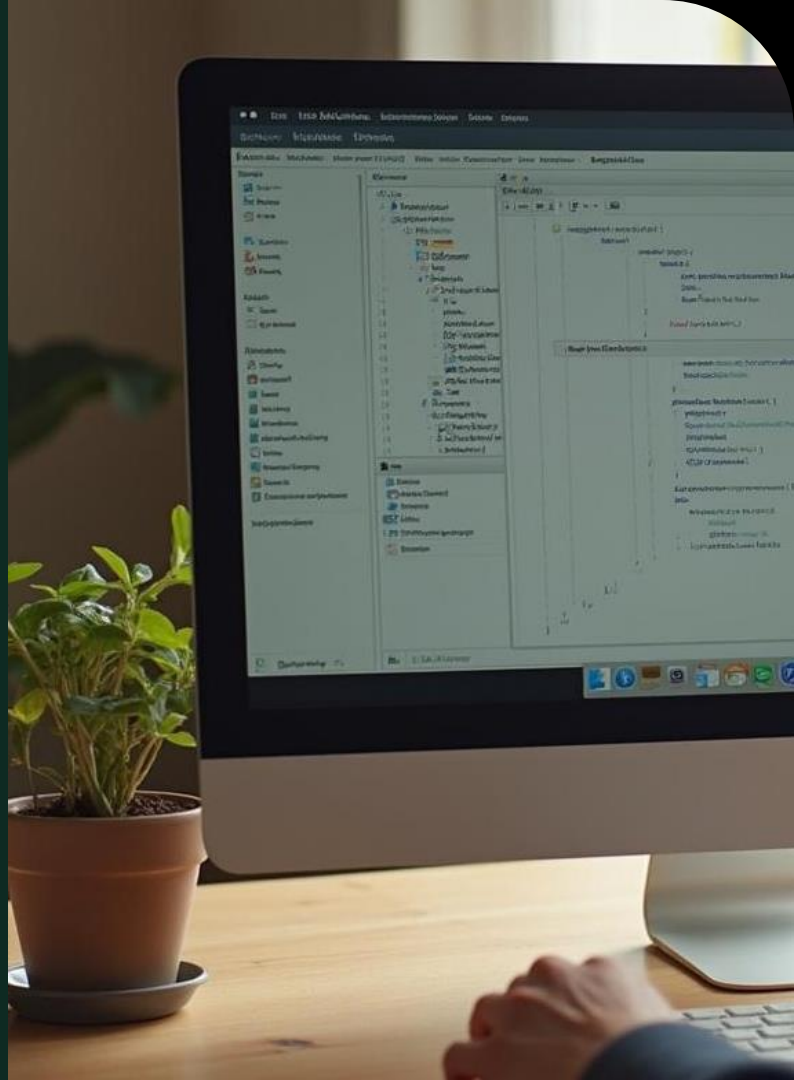

C:\Windows\system32\cmd.exe
icrosoft Windows [Version 10.0.19045.6456]
c) Microsoft Corporation. All rights reserved.
:\StudCRUD>cd backend
:\StudCRUD\backend>npm run dev
backend@1.0.0 dev
nodemon server.js
nodemon] 3.1.14
nodemon] to restart at any time, enter `rs`
nodemon] watching path(s): *.*
nodemon] watching extensions: js,mjs,cjs,json
nodemon] starting `node server.js`
injected env (1) from .env
onnected to MongoDB
erver running on port: 8000
C:\Windows\system32\cmd.exe
VITE v6.4.4 ready in 517 ms
Local: http://localhost:5173/
Network: use --host to expose
press h + enter to show help

# MongoDB Compass: Database Setup

MongoDB Compass lets you visually manage your database.

Connect to the local server, create a database named StudCRUD, and a students collection. Add student records manually through the UI in JSON format. This helps you understand document-based data storage compared to traditional relational databases.

↓



MongoDB Compass

Connections Edit View Help

Compass

My Queries

CONNECTIONS (2)

Search connections

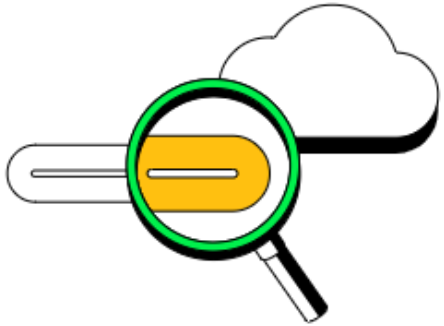
Atlas

my local machine cluster

Welcome

Welcome to MongoDB

To get started, connect to an existing serv



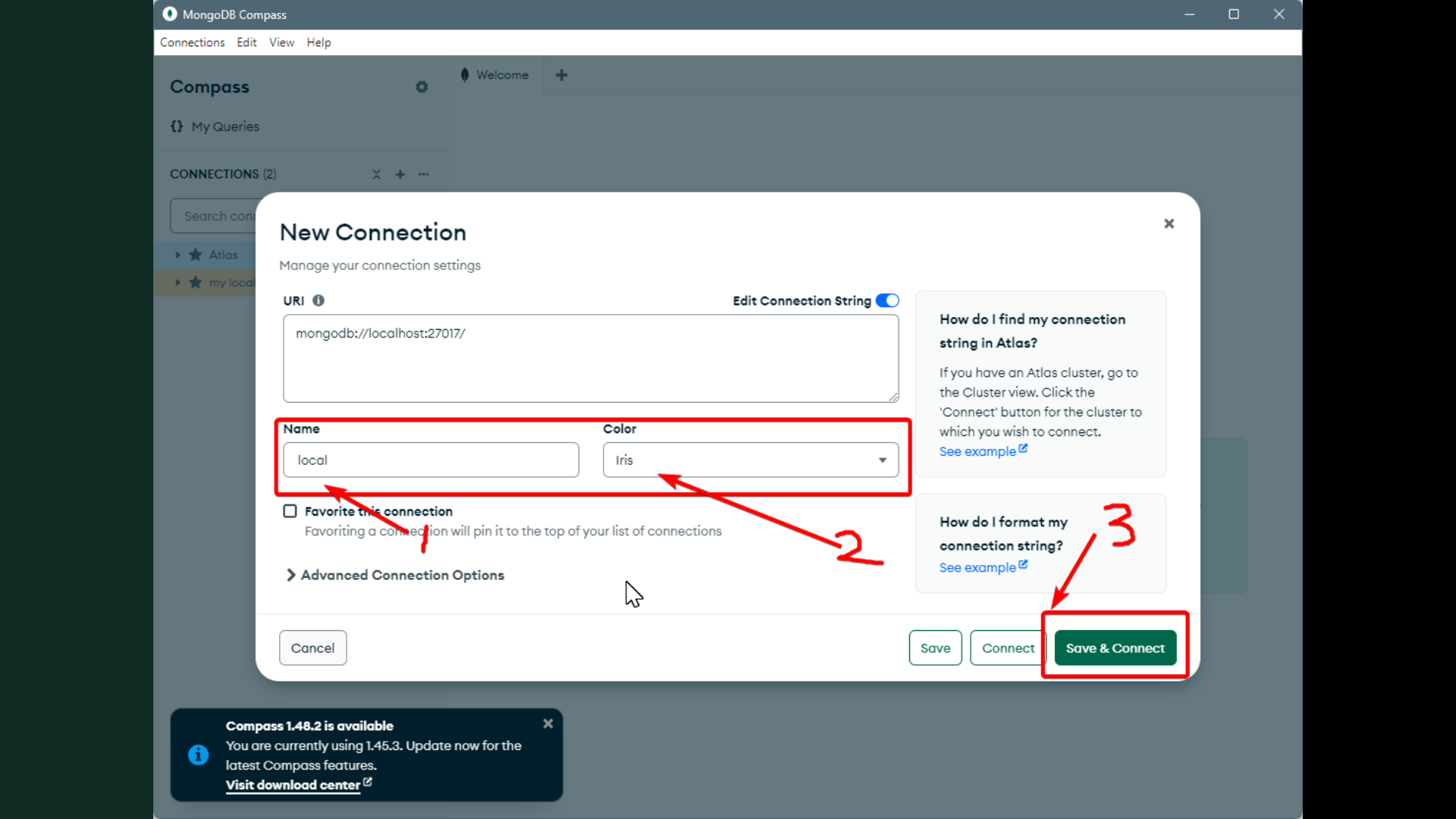
+ Add new connection

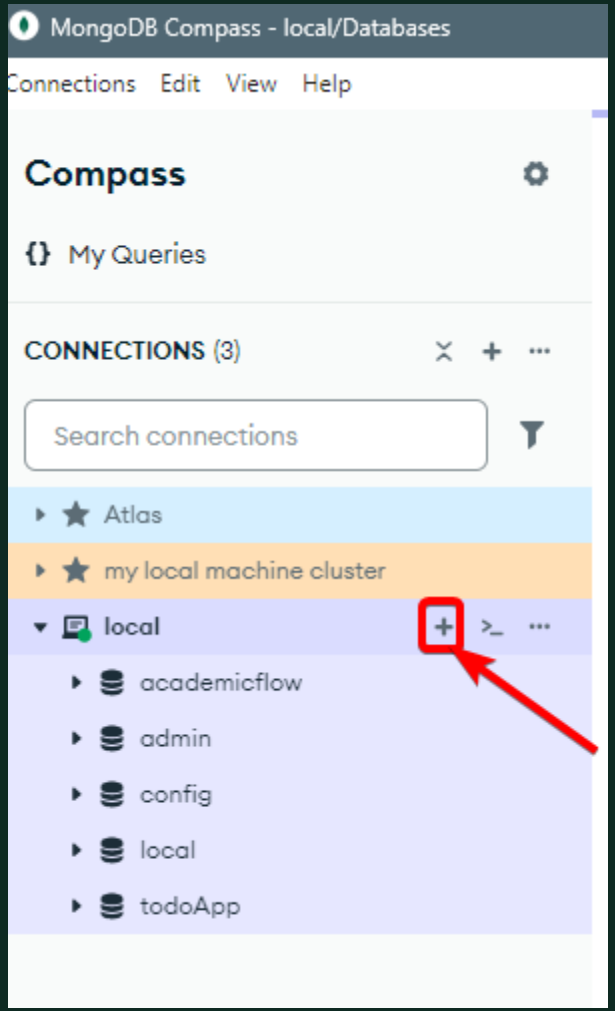
New to Compass and don't have a

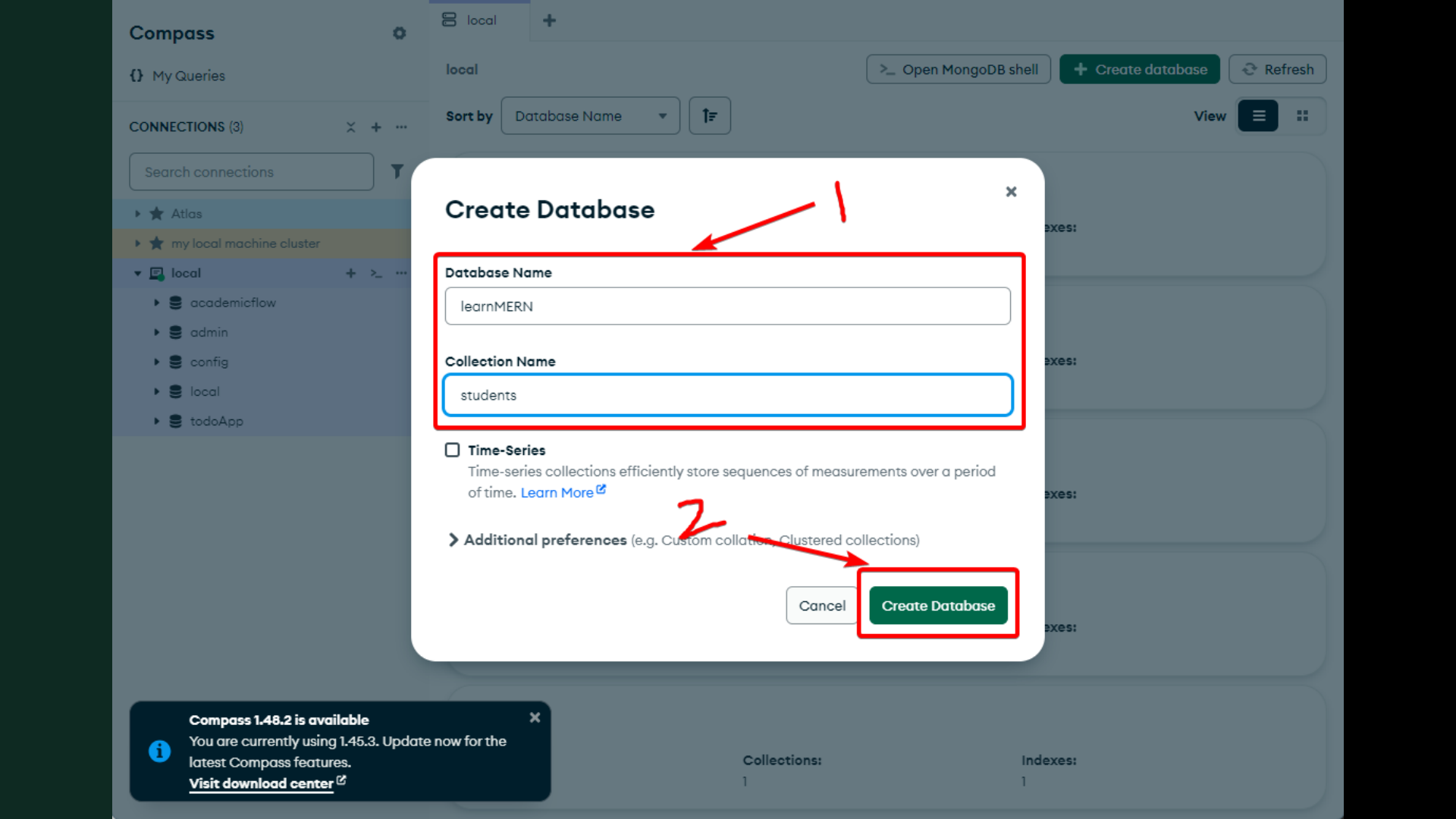
If you don't already have a cluster, yo

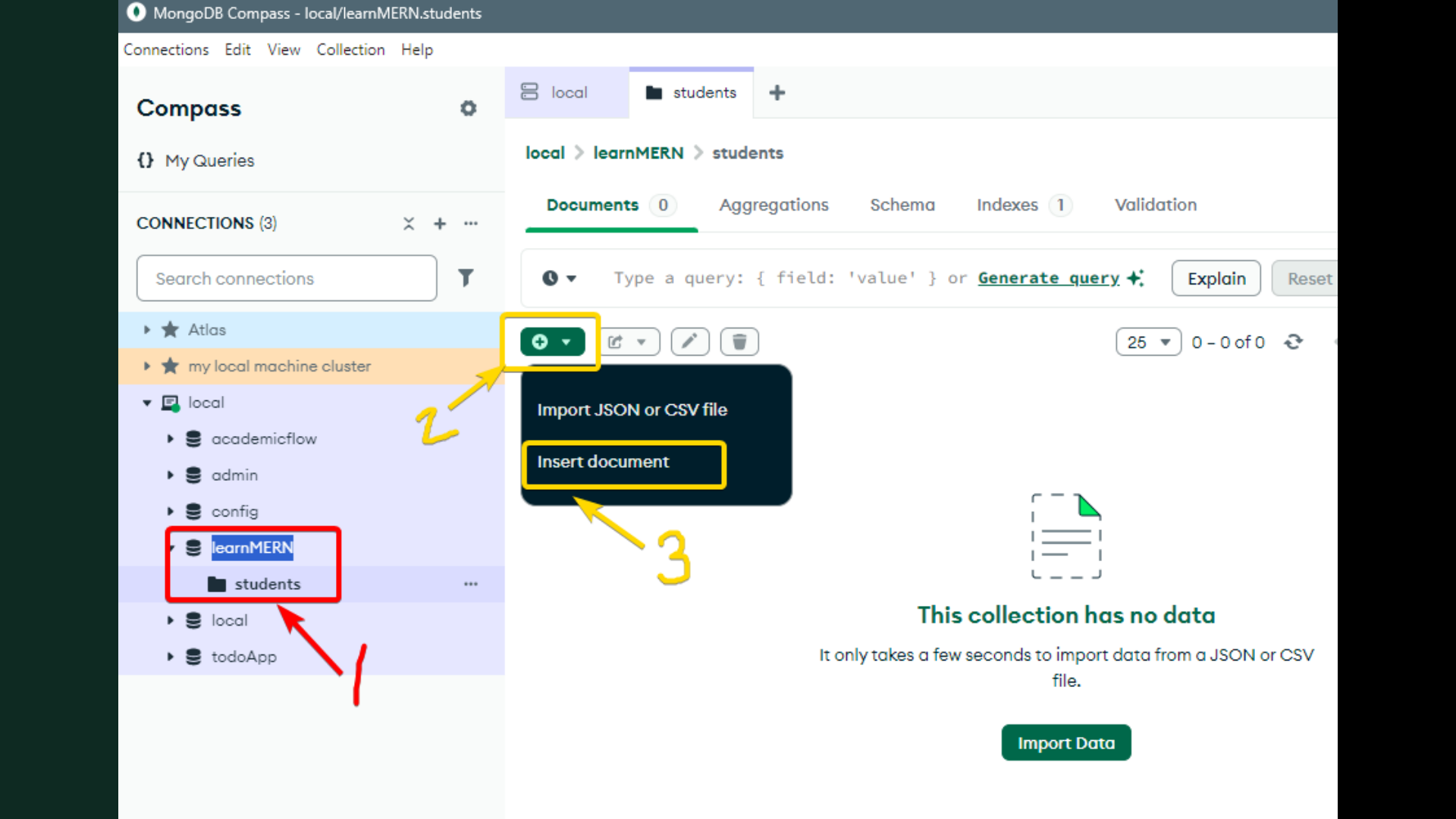
using MongoDB Atlas

CREATE FREE CLUSTER


MongoDB Compass
Connections Edit View Help
Compass
My Queries
CONNECTIONS (2)
Search con
Atlas
my local
Welcome
New Connection
Manage your connection settings
URI
Edit Connection String
mongodb://localhost:27017/
Name
local
Color
Iris
Favorite this connection
Favoriting a connection will pin it to the top of your list of connections
Advanced Connection Options
How do I find my connection string in Atlas?
If you have an Atlas cluster, go to the Cluster view. Click the 'Connect' button for the cluster to which you wish to connect.
See example
How do I format my connection string?
See example
1
2
3
Cancel
Save
Connect
Save & Connect
Compass 1.48.2 is available
You are currently using 1.45.3. Update now for the latest Compass features.
Visit download center


MongoDB Compass - local/Databases
Connections Edit View Help
Compass
My Queries
CONNECTIONS (3)
Search connections
Atlas
my local machine cluster
local
academicflow
admin
config
local
todoApp


Compass
My Queries
CONNECTIONS (3)
Search connections
Atlas
my local machine cluster
local
academicflow
admin
config
local
todoApp
local
Open MongoDB shell
Create database
Refresh
Sort by
Database Name
View
Create Database
1
Database Name
learnMERN
Collection Name
students
Time-Series
Time-series collections efficiently store sequences of measurements over a period of time. Learn More
2
Additional preferences (e.g. Custom collation, Clustered collections)
Cancel
Create Database
Compass 1.48.2 is available
You are currently using 1.45.3. Update now for the latest Compass features.
Visit download center
Collections:
1
Indexes:
1


MongoDB Compass - local/learnMERN.students
Connections Edit View Collection Help
Compass
My Queries
CONNECTIONS (3)
Search connections
Atlas
my local machine cluster
local
academicflow
admin
config
learnMERN
students
local
todoApp
local
students
local > learnMERN > students
Documents 0
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
25
0 – 0 of 0
Import JSON or CSV file
Insert document
1
2
3
This collection has no data
It only takes a few seconds to import data from a JSON or CSV file.
Import Data

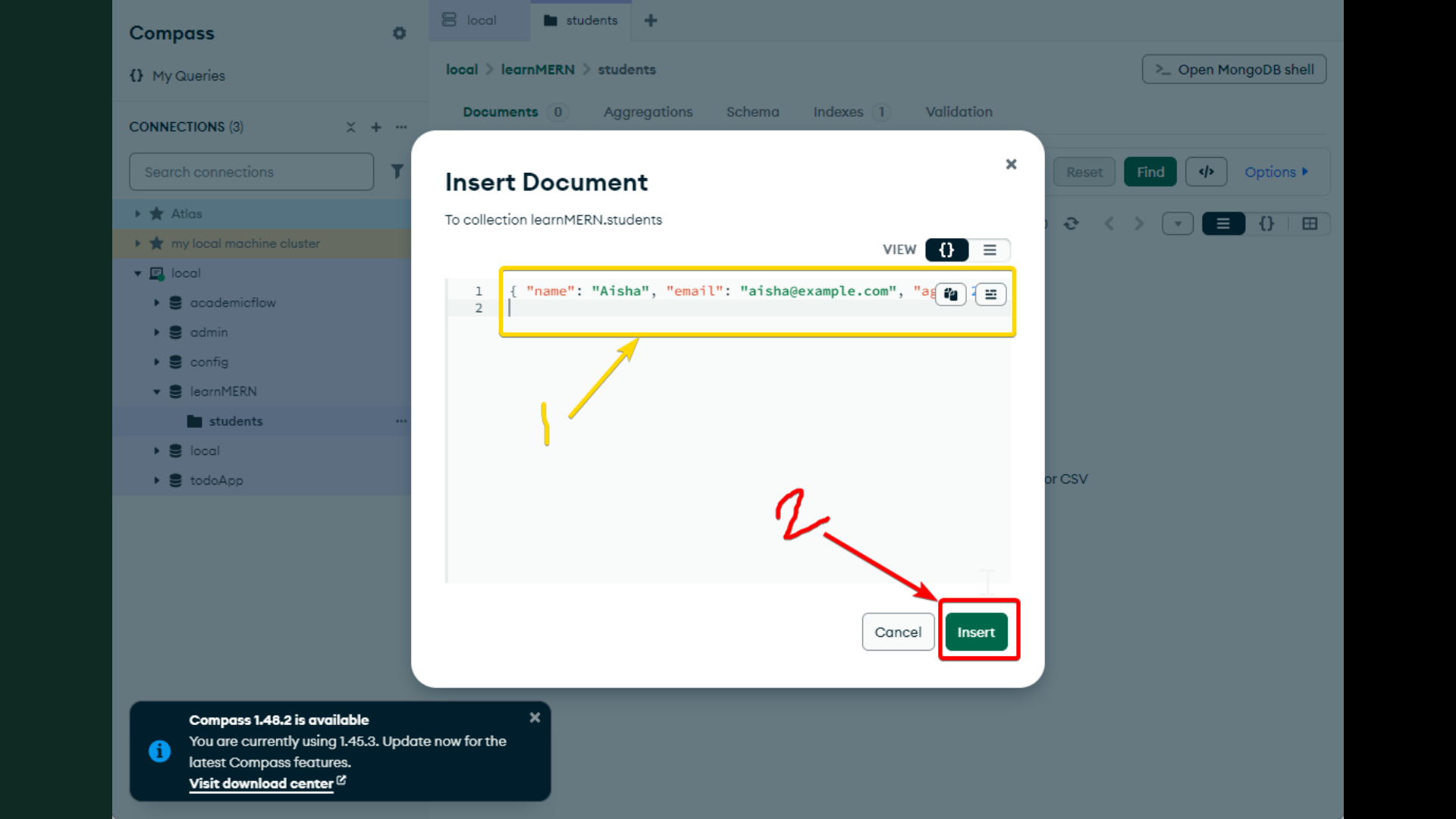
# Copy code from chapter 4 step 3 from my site

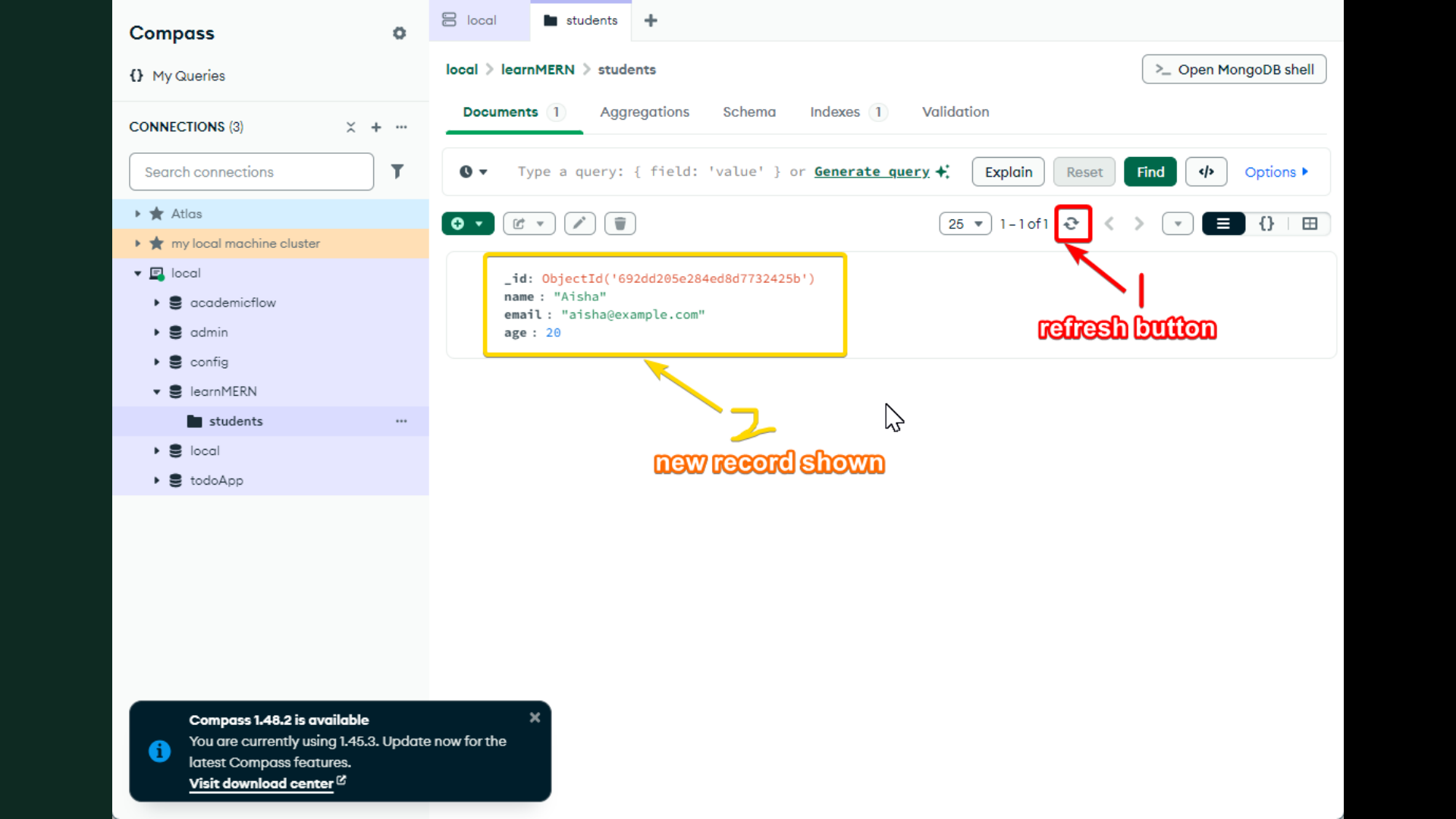
**Step 3: Insert records via GUI**

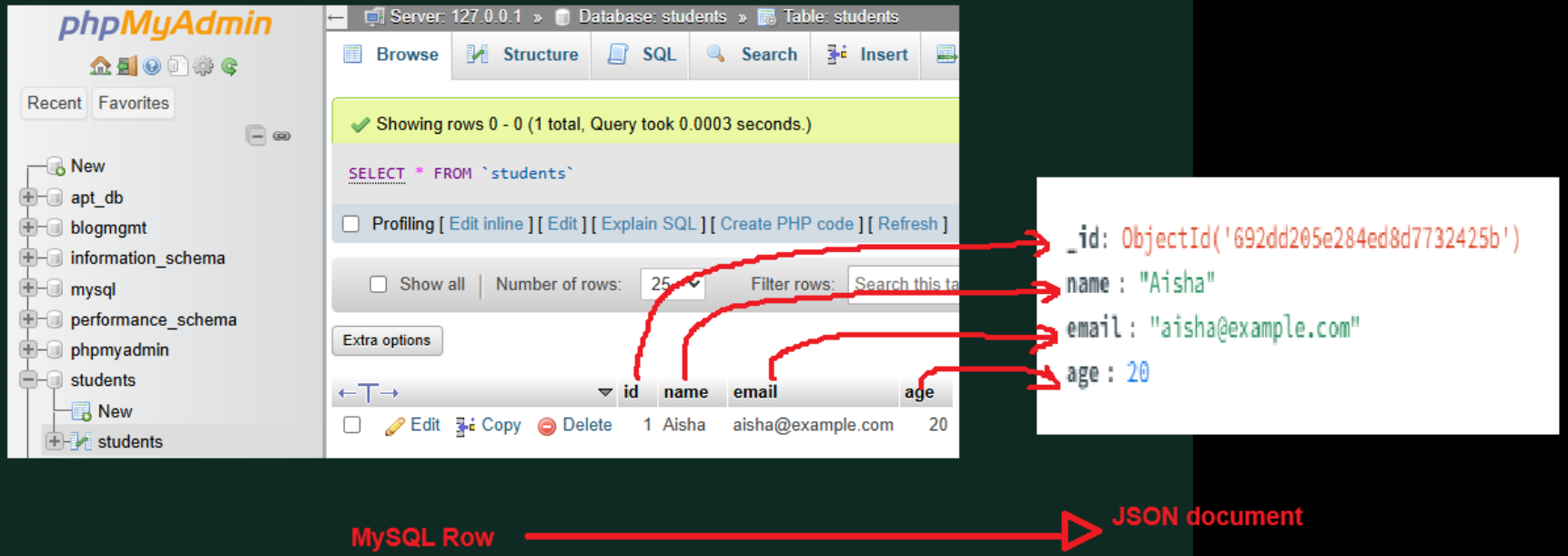
1. Open `Sidebar -> Local > learnMERN > students`.
2. From `Add Data` Click **Insert Document**.
3. Add JSON documents like:

Copy

```
{ "name": "Aisha", "email": "aisha@example.com", "age": 20 }
```


Compass
My Queries
CONNECTIONS (3)
Search connections
Atlas
my local machine cluster
local
academicflow
admin
config
learnMERN
students
local
todoApp
local
students
local > learnMERN > students
Open MongoDB shell
Documents 0
Aggregations
Schema
Indexes 1
Validation
Reset
Find
Options
Insert Document
To collection learnMERN.students
VIEW
1 { "name": "Aisha", "email": "aisha@example.com", "ag
2
1
2
Cancel
Insert
Compass 1.48.2 is available
You are currently using 1.45.3. Update now for the latest Compass features.
Visit download center


Compass
My Queries
CONNECTIONS (3)
Search connections
Atlas
my local machine cluster
local
academicflow
admin
config
learnMERN
students
local
todoApp
local
students
local > learnMERN > students
Open MongoDB shell
Documents 1
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
25
1 - 1 of 1
_id: ObjectId('692dd205e284ed8d7732425b')
name : "Aisha"
email : "aisha@example.com"
age : 20
refresh button
new record shown
Compass 1.48.2 is available
You are currently using 1.45.3. Update now for the latest Compass features.
Visit download center


phpMyAdmin
Recent
Favorites
New
apt_db
blogmgmt
information_schema
mysql
performance_schema
phpmyadmin
students
New
students
Server: 127.0.0.1 » Database: students » Table: students
Browse
Structure
SQL
Search
Insert
Showing rows 0 - 0 (1 total, Query took 0.0003 seconds.)
SELECT * FROM `students`
Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]
Show all
Number of rows:
25
Filter rows:
Search this ta
Extra options
id
name
email
age
Edit
Copy
Delete
1
Aisha
aisha@example.com
20
_id: ObjectId('692dd205e284ed8d7732425b')
name : "Aisha"
email : "aisha@example.com"
age : 20
MySQL Row
JSON document

## ✅ 1. PHP `$_POST` Associative Array Representation

**Example (what PHP receives):**

php Copy code

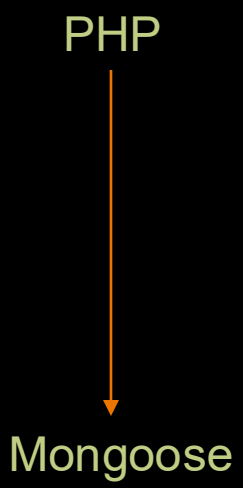
```php
$_POST = [
    "name"  => "Aisha",
    "email" => "aisha@example.com",
    "age"   => "20"
];
```

## ✅ 2. Mongoose JSON Representation

**Example (what Mongoose receives):**

json Copy code

```json
{
  "name": "Aisha",
  "email": "aisha@example.com",
  "age": 20
}
```



## ✨ Transmission Example (Node/Mongoose side)

### Client → Express JSON POST

javascript

```javascript
fetch("/students", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    name: "Aisha",
    email: "aisha@example.com",
    age: 20
  })
});
```

### Express endpoint

javascript

```javascript
app.post("/students", (req, res) => {
  console.log(req.body);
  // { name: 'Aisha', email: 'aisha@example.com', age: 20 }
});
```

### 🍼 Example: HTML Form

Html ^ Copy

```html
<form action="submit.php" method="post">
  <input type="text" name="name" value="Aisha" />
  <input type="email" name="email" value="aisha@example.com" />
  <input type="number" name="age" value="20" />
  <input type="submit" value="Send" />
</form>
```

### 🐘 PHP: Accessing `$_POST` as an Associative Array

Php ^ Copy

```php
<?php
// submit.php
print_r($_POST);
?>
```

**Output:**

Code ^ Copy

```
Array
(
    [name] => Aisha
    [email] => aisha@example.com
    [age] => 20
)
```

# MySQL to Mongo Term Correlation

- MySQL Rows → MongoDB JSON Documents
- MySQL Table → MongoDB Collection
- MySQL Database → MongoDB Database
- MySQL Table Structure → MongoDB Schema



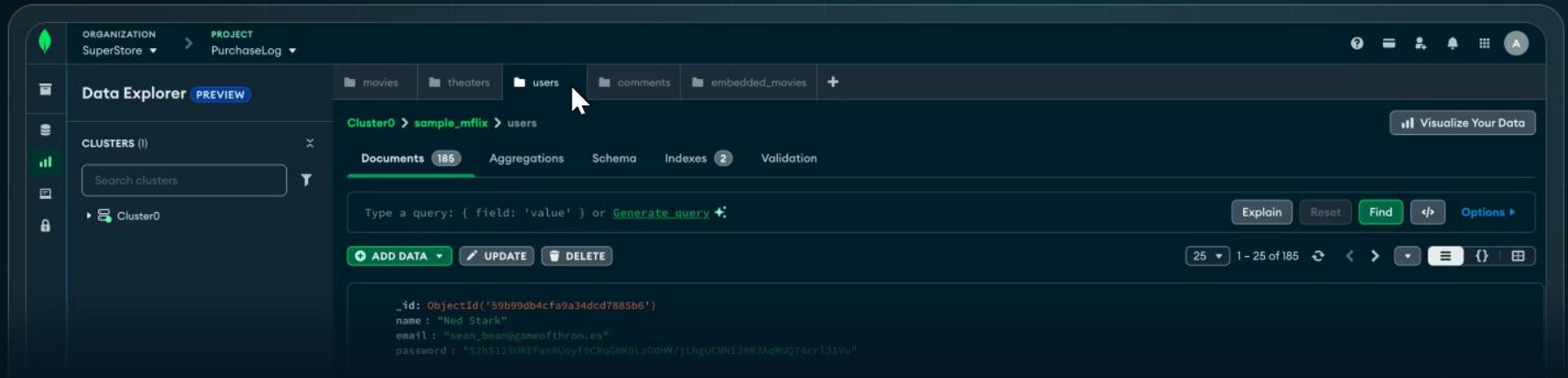
JOIN US Connect with our experts at re:Invent 2025 →

# The database for dynamic, demanding software

AI depends on fluid, instantly accessible data, not fragmented silos. Consolidate structured and unstructured data—text, video, audio, time series, and more—into a single, flexible system.

Get Started | Documentation

Ask MongoDB Assistant



# Connecting Mongo DB and Compass to Express Backend

→ Build your backend APIs by defining a Student schema with Mongoose for enforced structure. Unlike Traditional DB where a we define a table structure at table creation before inserting records. Mongo DB does not define any structure. This is because it is schema less and flexible.

# Connecting Mongo DB and Compass to Express Backend

→ This however means we have to tell the express backend in what format we are receiving data from mongodb because each row can store any type of data or any amount of fields. In the case of mysql we already define a permanent structure at table creation so php backend will recognise structure automatically

This is backend\server.js file showing server running code do not remove only add the new code in next step

```
import express from 'express';
import cors from 'cors';
import dotenv from 'dotenv';

dotenv.config(); // Load environment variables

const app = express();
const port = process.env.PORT || 8000; // Default port

app.use(cors()); // Enable CORS
app.use(express.json()); // Parse JSON bodies

app.listen(port, () => {
    console.log("Server running on port: " + port);
});
```

Chapter 5 step 0 on my site covers how we can tell express backend the structure of our students collection (table). Copy and paste the code in backend\server.js !! without replacing !! below pre-existing code

**Step 0: Connect to your database + specify collection structure**

Copy

```
import mongoose from 'mongoose';

// Connect to database 'learnMERN'
await mongoose.connect('mongodb://localhost:27017/learnMERN');

// Define schema
const studentSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true },
  age: { type: Number, required: true }
});

// Create model -> collection will be 'students'
const Student = mongoose.model('Student', studentSchema);

export default Student;
```

# Creating APIs with Express and Mongoose

→ Implement CRUD operations using Express and Mongoose with clearly defined routes for reading, creating, updating, and deleting student data. Use Mongoose methods such as find(), create(), findByIdAndUpdate(), and findByIdAndDelete() for database interactions. Test endpoints with curl or Postman, and verify database updates visually with MongoDB Compass to ensure accuracy and consistency in your application.

**Step 1: List students (Read)**

- Add a GET route in `server.js`: Copy

if url is /api/students and request is get then read operation must be called

```
app.get('/api/students', async (req, res) => {
  try {
    const students = await Student.find();
    res.json(students);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

read mongodb

X: 876 Y: 380

- **Test:** Open new cmd window and run the following: Copy

```
curl http://localhost:8000/api/students
```

- **Verify:** Matches the records inserted via Compass GUI.

**Step 2: Insert record (Create)**

- Add a POST route: Copy

if post request is made to /api/students then insertion operation is called

```
app.post('/api/students', async (req, res) => {
    try {
    const student = await Student.create(req.body);
    res.status(201).json("Record inserted");
    } catch (err) {
    res.status(400).json({ error: err.message });
    }
});
```

2

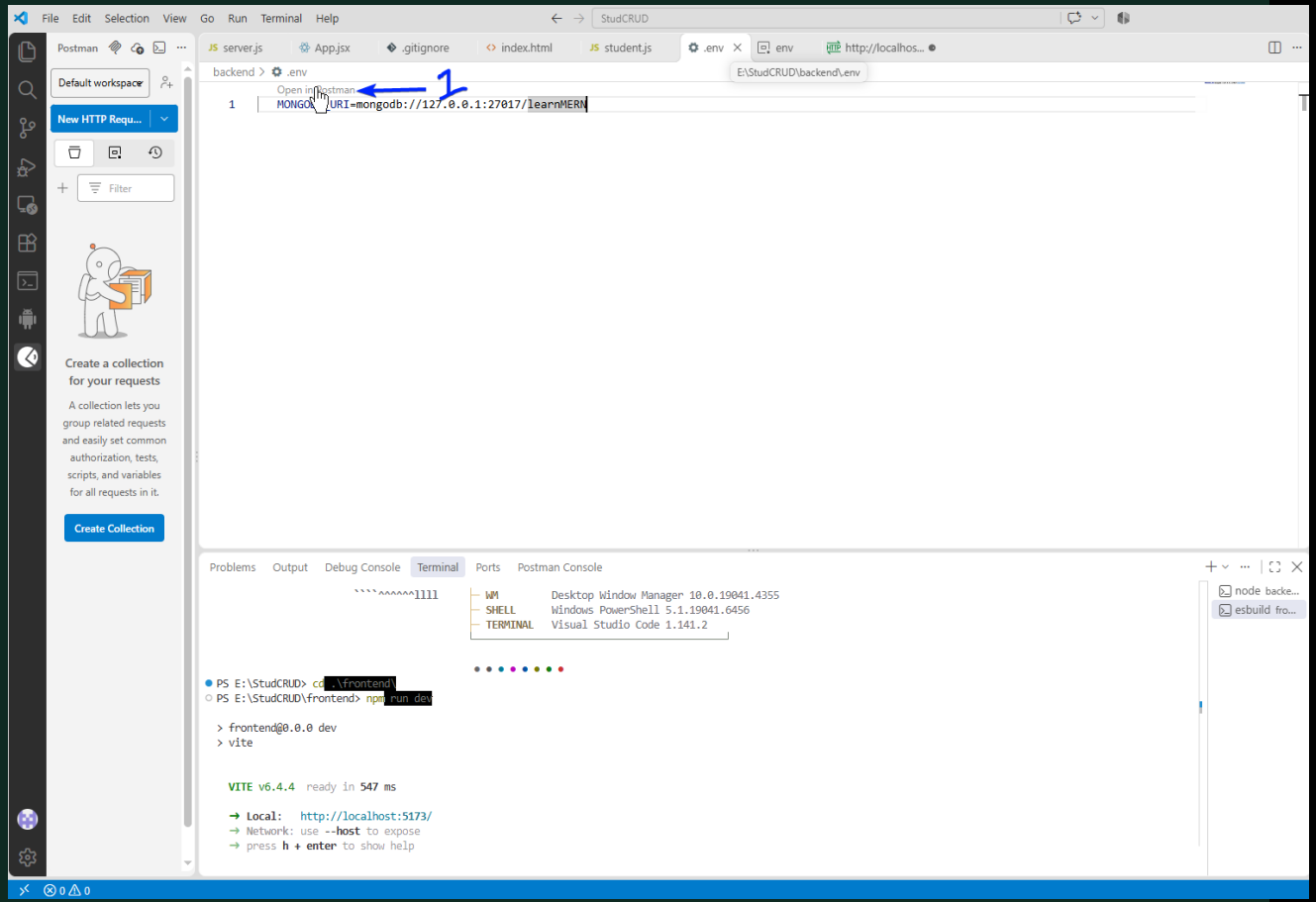
You can use this instead see the inserted record: `res.status(201).json(student);`

- **Test:** Open new cmd window and run the following: Copy

```
curl -X POST http://localhost:8000/api/students ^
-H "Content-Type: application/json" ^
-d "{\"name\":\"Shashi\",\"email\":\"shashi@example.com\",\"age\":22}"
```

- **Verify:** New record appears in Compass.

# Postman Method of Testing



File Edit Selection View Go Run Terminal Help

StudCRUD

Postman

Default workspace

New HTTP Request

2

Filter

Create a collection for your requests

A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.

Create Collection

server.js App.jsx .gitignore index.html student.js .env env http://localhos...

env

Save Share

Filter variables

| Variable | Type | Initial value | Current value |
|---|---|---|---|
| MONGODB_URI | default | mongodb://127.0.0.1:27017/learnMERN | mongodb://127.0.0.1:27017/learnMERN |
| Add new variable | | | |

Problems Output Debug Console Terminal Ports Postman Console

```
````^^^^^^1111
    WM        Desktop Window Manager 10.0.19041.4355
    SHELL     Windows PowerShell 5.1.19041.6456
    TERMINAL  Visual Studio Code 1.141.2

PS E:\StudCRUD> cd .\frontend\
PS E:\StudCRUD\frontend> npm run dev

> frontend@0.0.0 dev
> vite


  VITE v6.4.4  ready in 547 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

node backe...

esbuild fro...
```

File Edit Selection View Go Run Terminal Help

StudCRUD

Postman

Default workspace

New HTTP Request

Filter

Create a collection for your requests

A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.

Create Collection

server.js | App.jsx | .gitignore | index.html | student.js | .env | env | http://localhos...

http://localhost:8000/api/students/

Save | env

POST 1 | http://localhost:8000/api/students/ 2 | Send 8

Params | Authorization | Headers (9) | 3 Body | Scripts | Settings | Code | Cookies

none | form-data | x-www-form-urlencoded | 6 raw | binary | GraphQL | JSON 5 | Beautify

4

```
{
"name":"shashy",
"email":"amit@jrs.com",
"age":35
}
```

insertion data

Body | Cookies | Headers (8) | Test Results

Status: 201 Created Time: 36 ms Size: 289 B

Pretty | Raw | Preview 7

"Record inserted" 9

Problems | Output | Debug Console | Terminal | Ports | Postman Console

```
````^^^^^^1111
    WM        Desktop Window Manager 10.0.19041.4355
    SHELL     Windows PowerShell 5.1.19041.6456
    TERMINAL  Visual Studio Code 1.141.2

PS E:\StudCRUD> cd .\frontend\
PS E:\StudCRUD\frontend> npm run dev

> frontend@0.0.0 dev
> vite

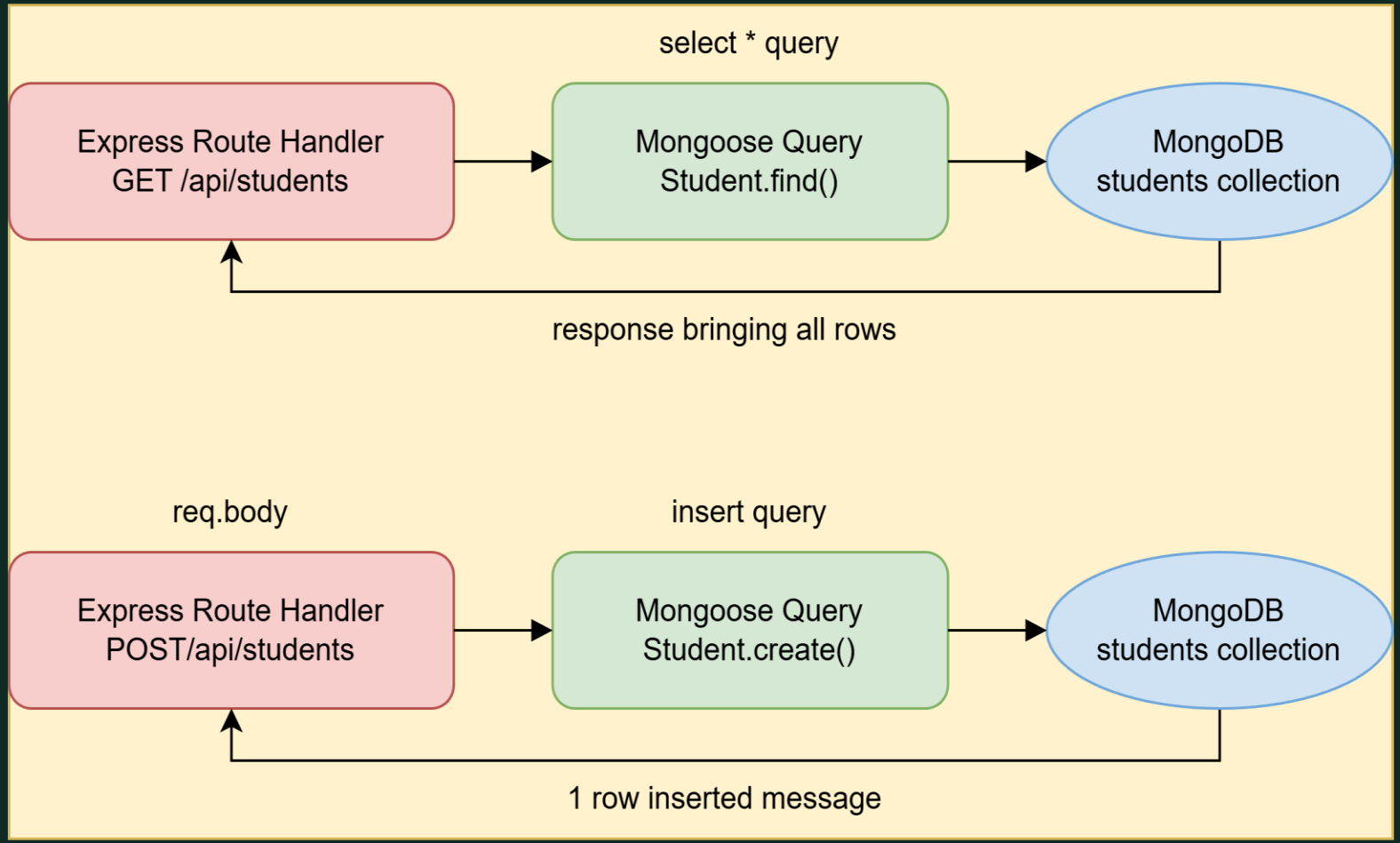
  VITE v6.4.4  ready in 547 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

node backe...

esbuild fro...
```


Student Form
Name:
devadas
Age
30
Email
devadas@gmail.com
Add/Update
{
"_id": {
"$oid": "692dd205e284ed8d7732425b"
},
"name": "Aisha",
"email": "aisha@example.com",
"age": 20
}
req.params.id
req.body
PUT
GET
POST
DELETE


select * query
Express Route Handler
GET /api/students
Mongoose Query
Student.find()
MongoDB
students collection
response bringing all rows
req.body
insert query
Express Route Handler
POST/api/students
Mongoose Query
Student.create()
MongoDB
students collection
1 row inserted message

**Step 4: Delete record (Delete)**

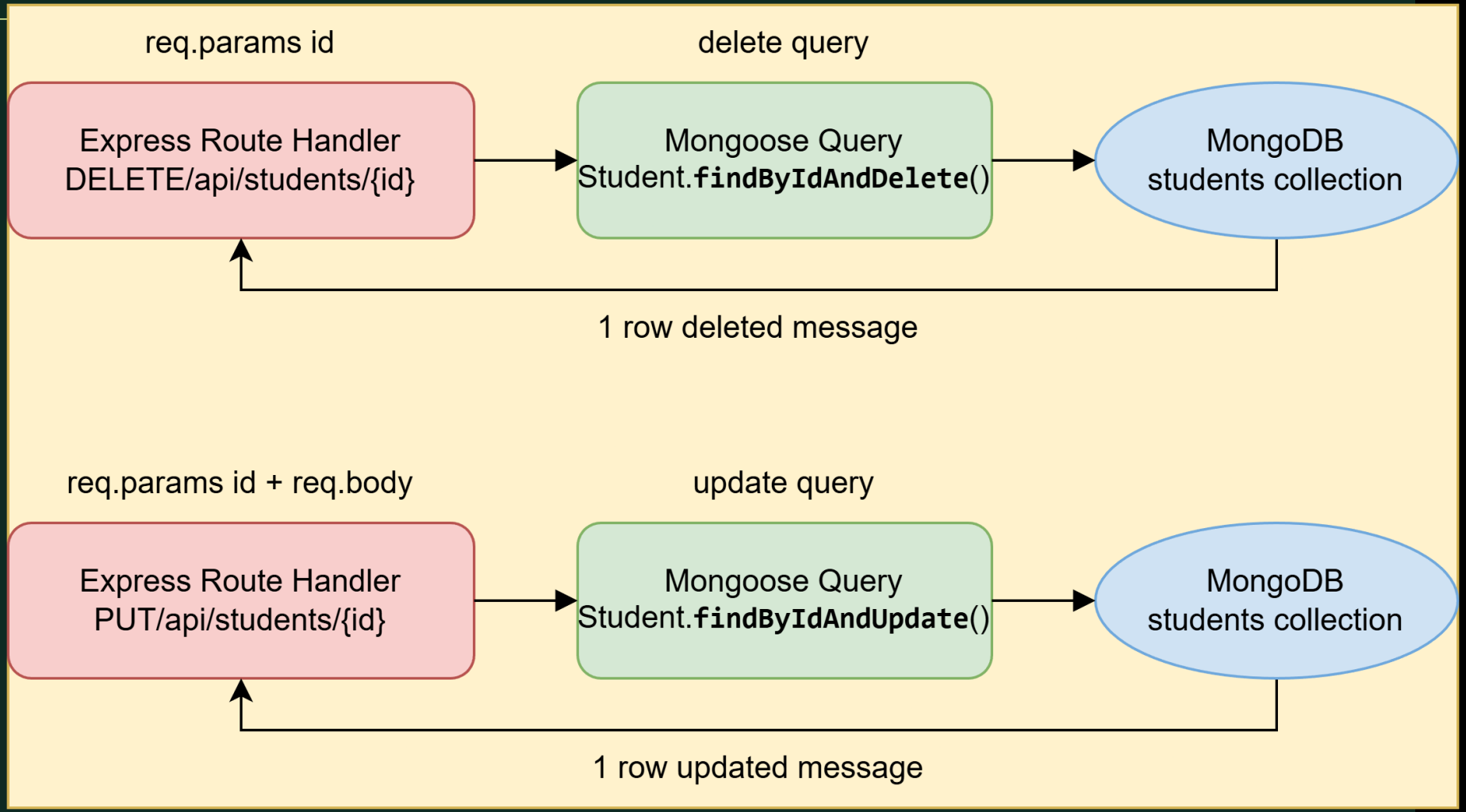
- Add a DELETE route: Copy




- **Test:** Open new cmd window and run the following: Copy

```
curl -X DELETE http://localhost:8000/api/students/692850edf9d3c59a5694b299
```

- **Verify:** Document disappears from Compass.


req.params id
delete query
Express Route Handler
DELETE/api/students/{id}
Mongoose Query
Student.findByIdAndDelete()
MongoDB
students collection
1 row deleted message
req.params id + req.body
update query
Express Route Handler
PUT/api/students/{id}
Mongoose Query
Student.findByIdAndUpdate()
MongoDB
students collection
1 row updated message

# Frontend Development with React

Copy code from chapter 6 step 1 into frontend\App.js to perform read operation at frontend and print list of students

↓



```jsx
import React, { useState, useEffect } from "react";

function App() {
  const [students, setStudents] = useState([]);

  // Load students on initial render
  useEffect(() => {
    fetch("http://localhost:8000/api/students")
      .then(res => res.json())
      .then(data => setStudents(data));
  }, []);

  return (
    <div style={{ margin: "2rem", fontFamily: "Arial, sans-serif" }}>
      <h1>🐍 Student Management</h1>

      {/* Student List */}
      <ul>
        {students.map(s => (
          <li key={s._id}>
            {s.name} ({s.age}) - {s.email}
          </li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```

App is the entire component

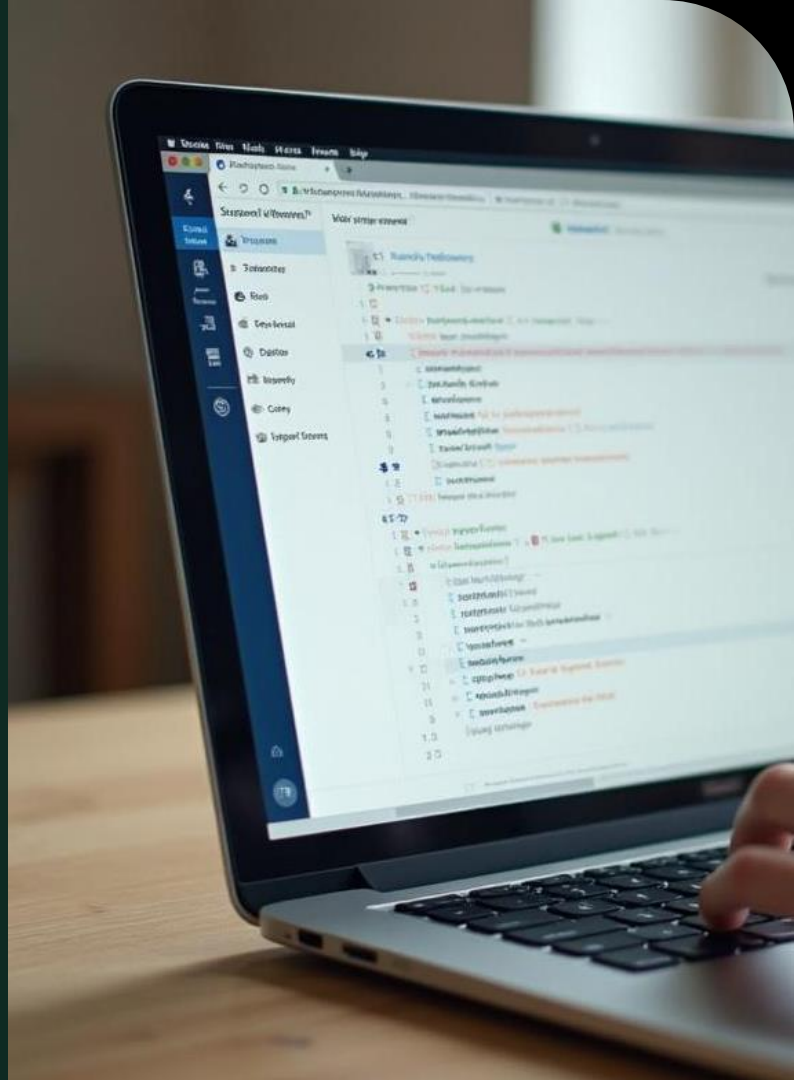
functional part

design part

# Fetching and Showing Student Data

Use React hooks like *useState* to store students and *useEffect* to fetch data on component mount. Fetch the list of students from the backend API and render them dynamically in a list. This initiates your React frontend connection to the backend and displays real-time student data on the UI with easy-to-understand state management techniques.

↓



**Code to paste (App.js)**

Copy

paste below already existing code

```
import React, { useState, useEffect } from "react";

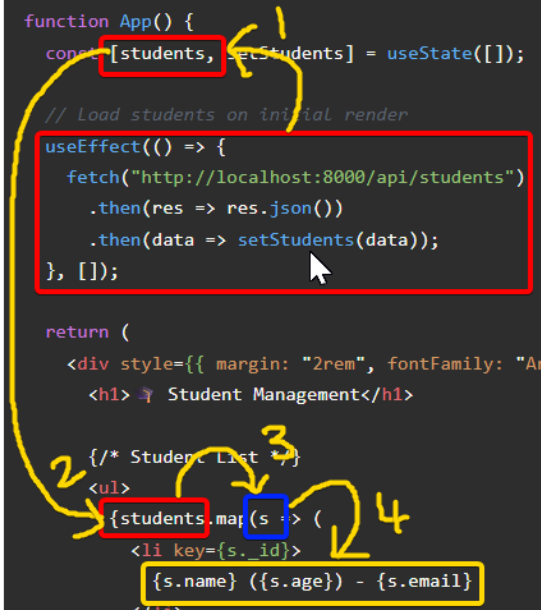
function App() {
  const [students, setStudents] = useState([]);

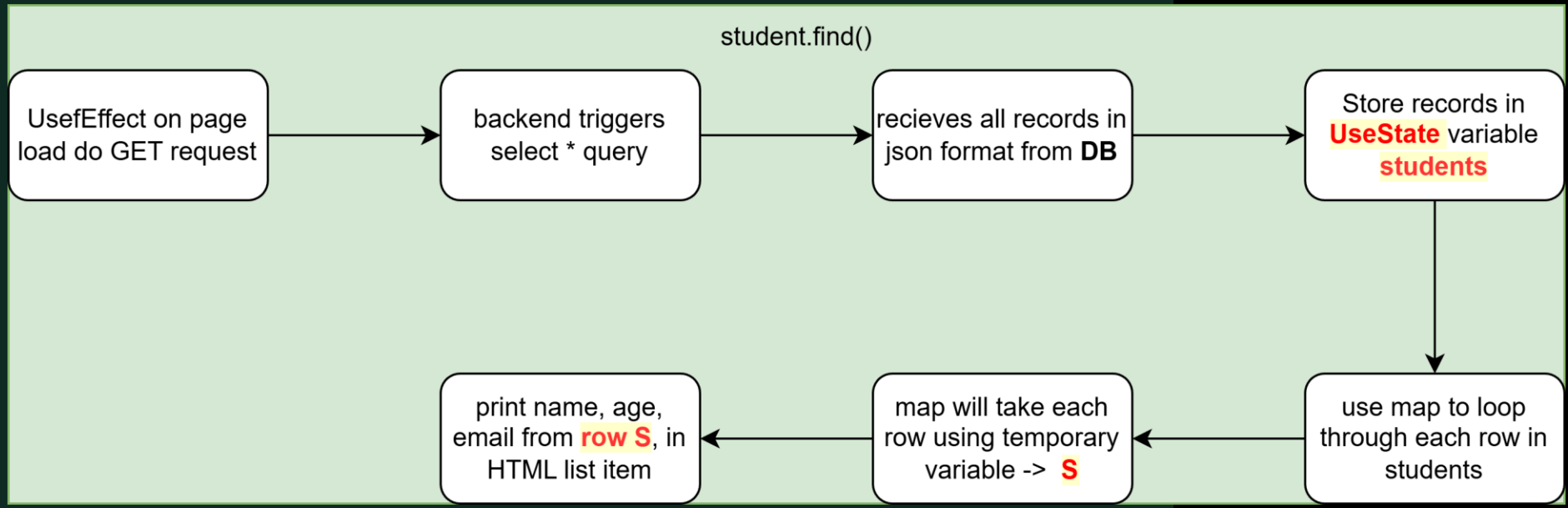
  // Load students on initial render
  useEffect(() => {
    fetch("http://localhost:8000/api/students")
      .then(res => res.json())
      .then(data => setStudents(data));
  }, []);

  return (
    <div style={{ margin: "2rem", fontFamily: "Arial, sans-serif" }}>
      <h1>🎓 Student Management</h1>

      {/* Student List */}
      <ul>
        {students.map(s => (
          <li key={s._id}>
            {s.name} ({s.age}) - {s.email}
          </li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```




student.find()
UsefEffect on page load do GET request
backend triggers select * query
recieves all records in json format from DB
Store records in UseState variable students
use map to loop through each row in students
map will take each row using temporary variable -> S
print name, age, email from row S, in HTML list item

# Deleting Records

→ Pass id through a button and make DELETE request

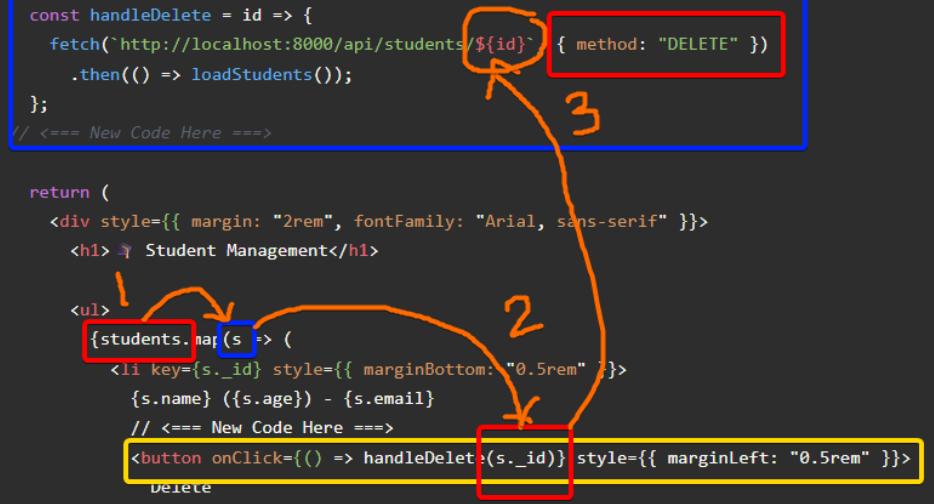
```
  const loadStudents = () => {
    fetch("http://localhost:8000/api/students")
      .then(res => res.json())
      .then(data => setStudents(data));
  };

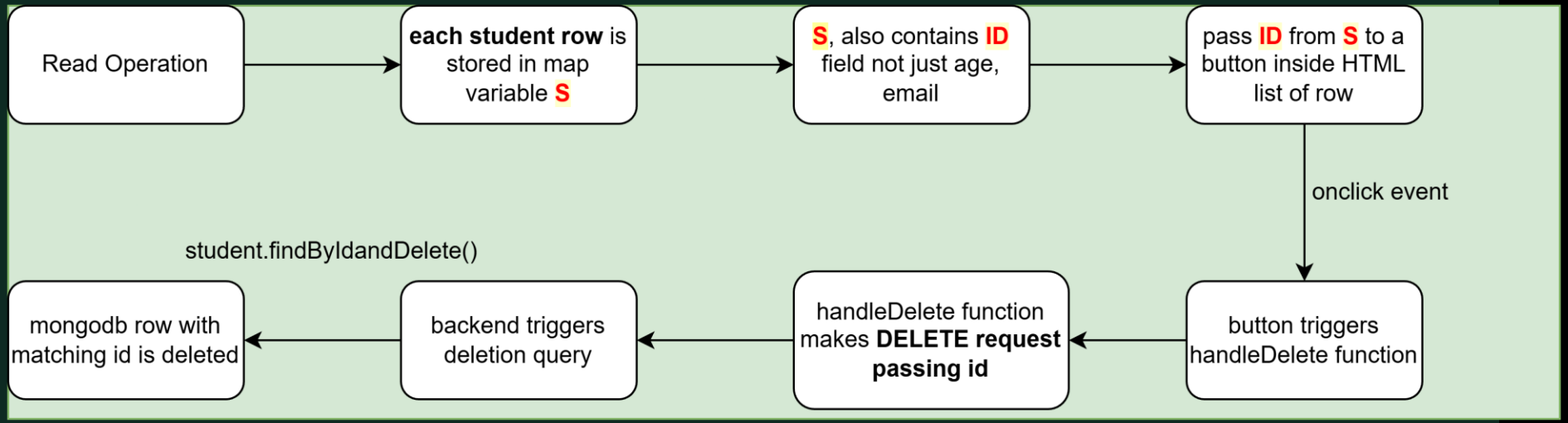
  useEffect(() => {
    loadStudents();
  }, []);

// <=== New Code Here ===>
  const handleDelete = id => {
    fetch(`http://localhost:8000/api/students/${id}`, { method: "DELETE" })
      .then(() => loadStudents());
  };
// <=== New Code Here ===>

  return (
    <div style={{ margin: "2rem", fontFamily: "Arial, sans-serif" }}>
      <h1>🎓 Student Management</h1>

      <ul>
        {students.map(s => (
          <li key={s._id} style={{ marginBottom: "0.5rem" }}>
            {s.name} ({s.age}) - {s.email}
            // <=== New Code Here ===>
            <button onClick={() => handleDelete(s._id)} style={{ marginLeft: "0.5rem" }}>
              Delete
            </button>
            // <=== New Code Here ===>
          </li>
        ))}
      </ul>
    </div>
  );
}
```




Read Operation
each student row is stored in map variable S
S, also contains ID field not just age, email
pass ID from S to a button inside HTML list of row
onclick event
button triggers handleDelete function
handleDelete function makes DELETE request passing id
backend triggers deletion query
student.findByIdandDelete()
mongodb row with matching id is deleted

# Creating and Updating Records

→ Add a controlled form to handle new student creation using *useState* for form inputs and track current editingID. Submitting the form triggers a POST request to the backend if editingID is null otherwise PUT request, followed by refreshing the student list. Implement update functionality with buttons that fill form inputs with pre existing data. After each mutation, reload the list to keep the UI synced with the backend.

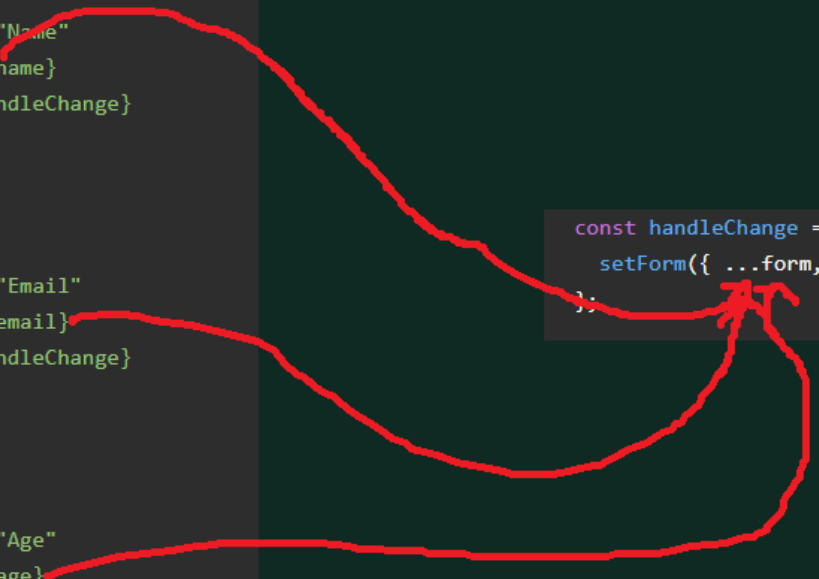
# Editing and Updating Records

→ Enable edit mode by storing the current records ID to toggle between create and update operations. Prefill the form fields with selected student data, modifying the submit button text accordingly. On form submission, send PUT requests to update existing records, then refresh the student list to show the latest data and maintain a seamless user experience.

# Insertion pt 1

```jsx
<input
  name="name"
  placeholder="Name"
  value={form.name}
  onChange={handleChange}
  required
/>
<input
  name="email"
  placeholder="Email"
  value={form.email}
  onChange={handleChange}
  required
/>
<input
  name="age"
  placeholder="Age"
  value={form.age}
  onChange={handleChange}
  required
/>
```

```jsx
const handleChange = e => {
  setForm({ ...form, [e.target.name]: e.target.value });
};
```



# Insertion pt 2

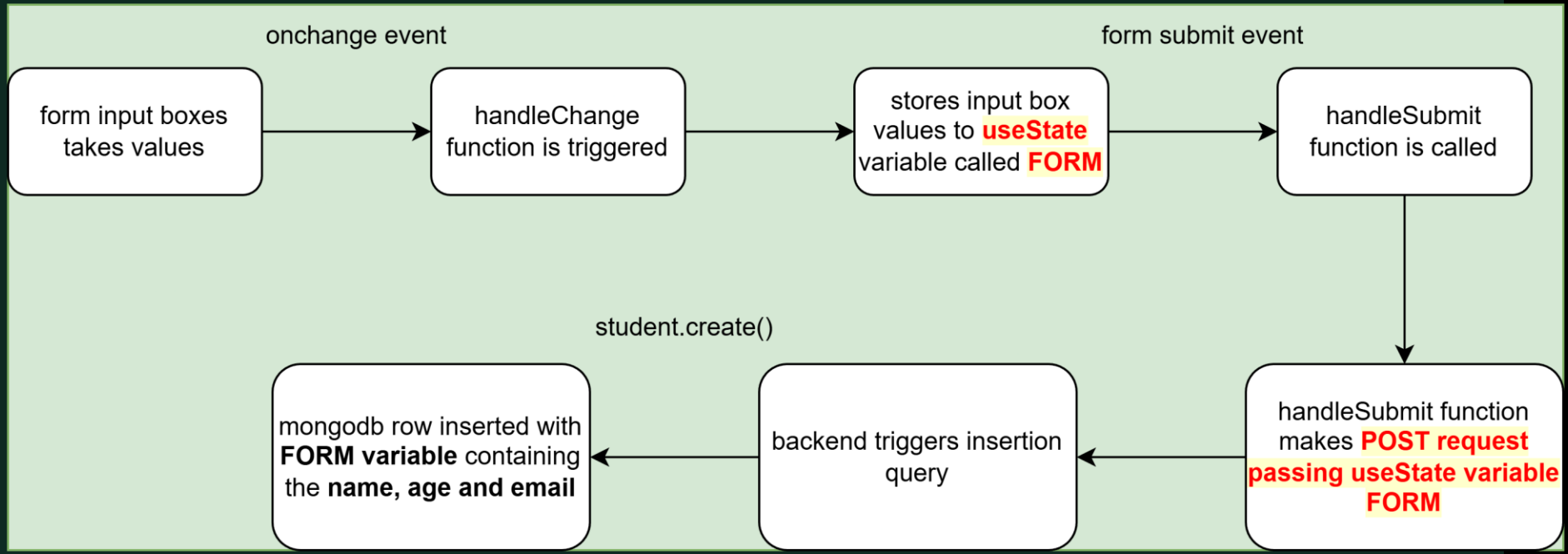
```
// <=== New Code Here ===>
  const handleChange = e => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };


  const handleSubmit = e => {
    e.preventDefault();
    fetch("http://localhost:8000/api/students", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(form)
    })
      .then(res => res.json())
      .then(() => {
        loadStudents();                 // refresh after create
        setForm({ name: "", email: "", age: "" }); // reset form
      });
  };
// <=== New Code Here ===>

  const handleDelete = id => {
    fetch(`http://localhost:8000/api/students/${id}`, { method: "DELETE" })
      .then(() => loadStudents());
  };

  return (
    <div style={{ margin: "2rem", fontFamily: "Arial, sans-serif" }}>
      <h1>🎓 Student Management</h1>

      // <=== New Code Here ===>
      {/* Form */}
      <form onSubmit={handleSubmit} style={{ marginBottom: "1rem" }}>
        <input
          name="name"
          placeholder="Name"
```


onchange event
form submit event
form input boxes takes values
handleChange function is triggered
stores input box values to useState variable called FORM
handleSubmit function is called
student.create()
handleSubmit function makes POST request passing useState variable FORM
backend triggers insertion query
mongodb row inserted with FORM variable containing the name, age and email

# Update pt 1 (showing editing record data in form input boxes and set value to editingID)

```jsx
{/* Student List */}
<ul>
  {students.map(s => (
    <li key={s._id} style={{ marginBottom: "0.5rem" }}>
      {s.name} ({s.age}) - {s.email}

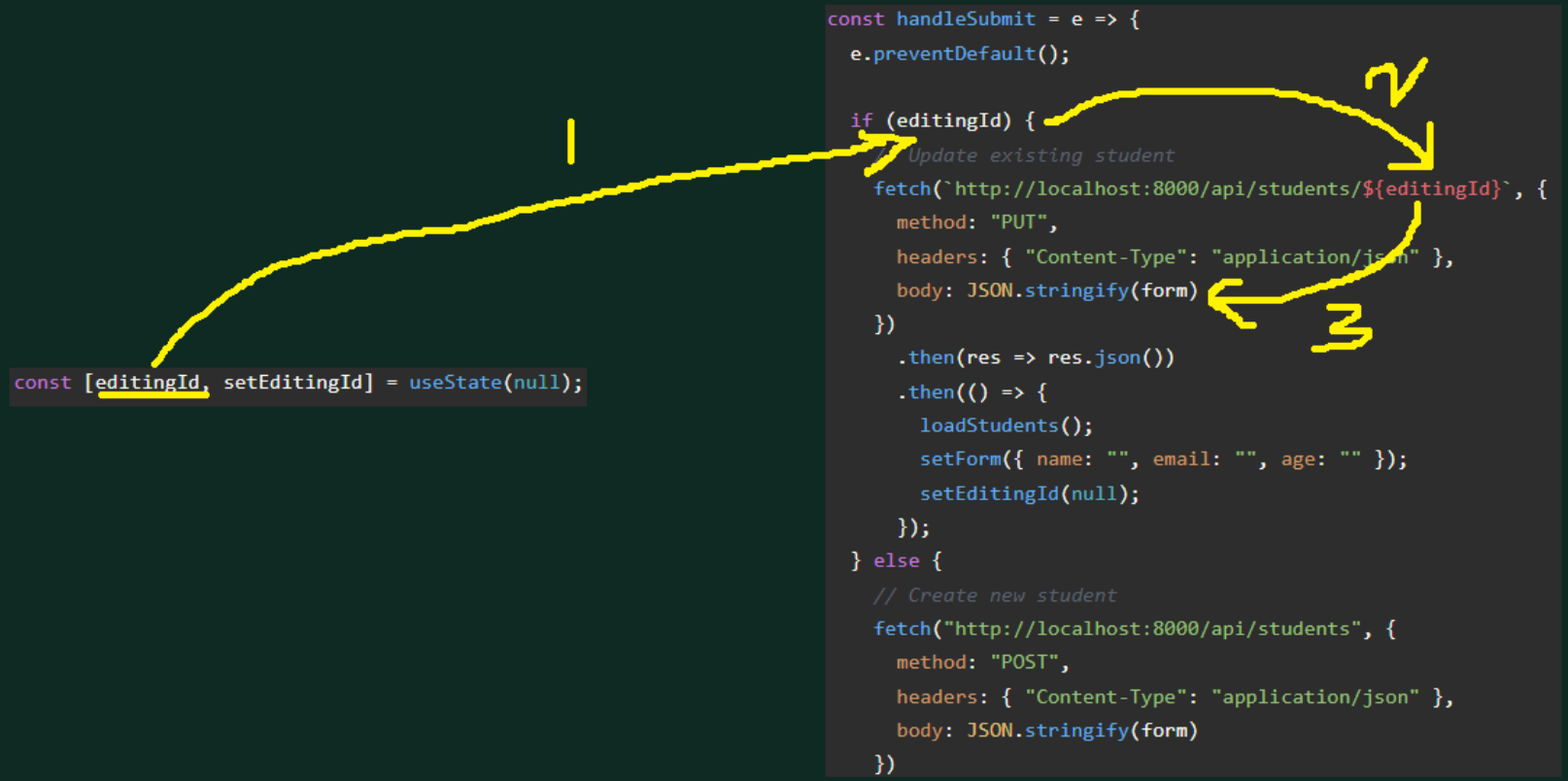
      // <=== New Code Here ===>
      <button onClick={() => handleEdit(s)} style={{ marginLeft: "1rem" }}>
        Edit
      </button>
      // <=== New Code Here ===>

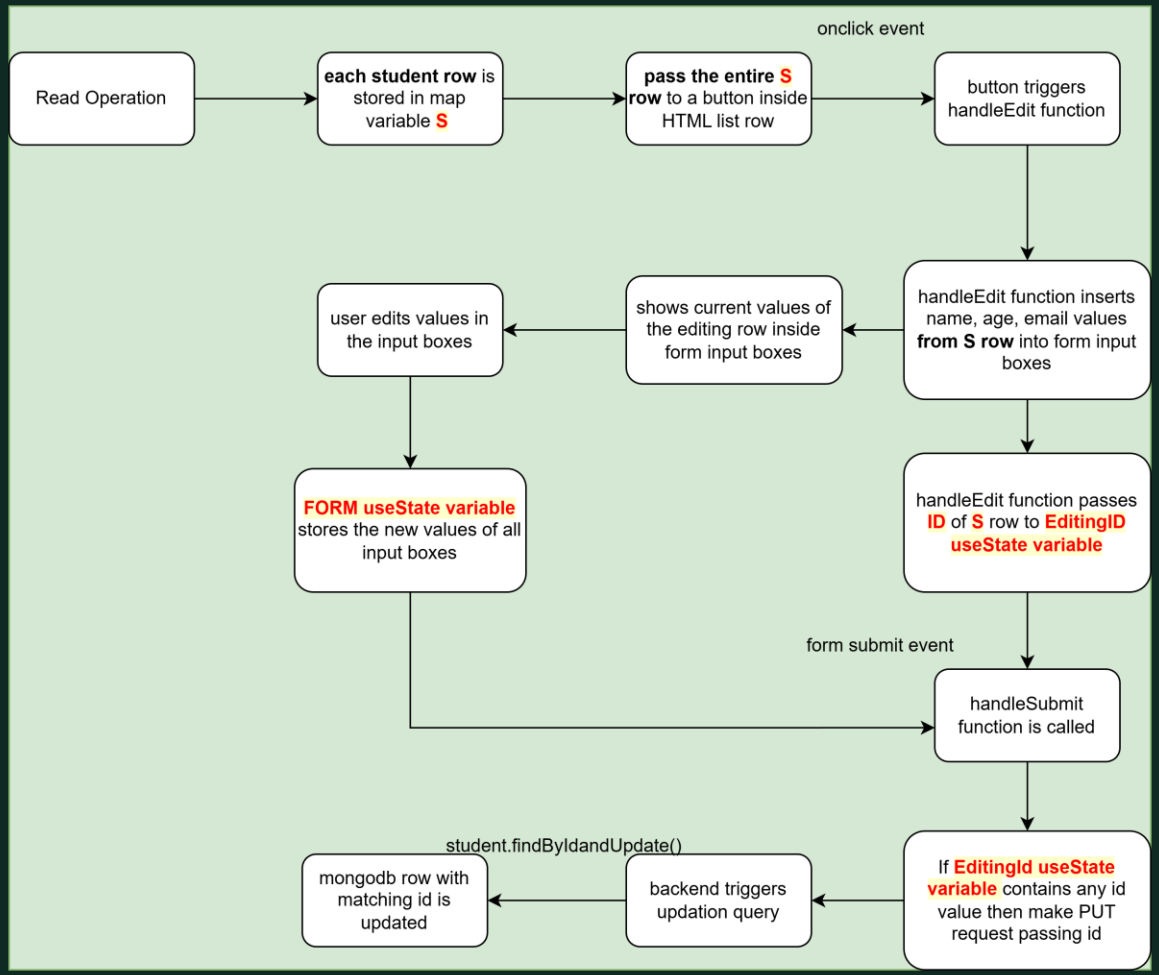
      <button onClick={() => handleDelete(s._id)} style={{ marginLeft: "0.5rem" }}>
        Delete
      </button>
    </li>
  ))}
</ul>
```

```jsx
const handleEdit = student => {
  setForm({ name: student.name, email: student.email, age: student.age });
  setEditingId(student._id);
};
```

```jsx
const [editingId, setEditingId] = useState(null);
```

# Update pt 2 (triggering PUT request if editingID has value)




Read Operation
each student row is stored in map variable S
pass the entire S row to a button inside HTML list row
onclick event
button triggers handleEdit function
handleEdit function inserts name, age, email values from S row into form input boxes
shows current values of the editing row inside form input boxes
user edits values in the input boxes
FORM useState variable stores the new values of all input boxes
handleEdit function passes ID of S row to EditingID useState variable
form submit event
handleSubmit function is called
If EditingId useState variable contains any id value then make PUT request passing id
backend triggers updation query
student.findByIdandUpdate()
mongodb row with matching id is updated

# Managing Environment Configuration

Organize environment variables in .env files using *dotenv* to keep sensitive configurations out of code. Ensure correct paths and ports for backend and frontend are defined consistently to avoid conflicts. Proper environment management facilitates smooth transitions between development, testing, and production setups.

↓



# Streamlining Development Workflow

→ Utilize tools like *nodemon* for automatic backend server restarts and Vite for fast React frontend builds. Run backend and frontend servers concurrently in separate command prompts for efficient development. Constantly reload data after CRUD operations to keep UI and database in sync, improving productivity and minimizing errors.

# Conclusions

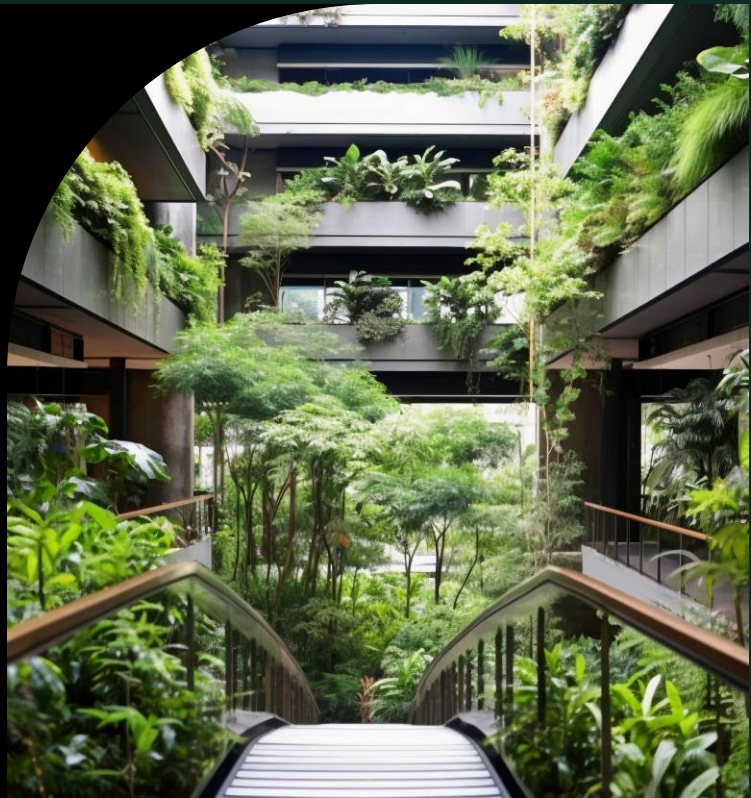
Building a MERN stack app involves setting up a robust backend with Express and Mongoose, managing data using MongoDB, and crafting an interactive React frontend. Understanding environment setup, troubleshooting, and smooth workflows sets the foundation for productive development. Mastering CRUD operations and syncing frontend-backend interactions completes the learning journey toward full-stack JavaScript proficiency.

↓



# Thanking All Joined Members

# Fin



Slidesgo